

Groupby_pandas

August 18, 2025

1 Group by in pandas

```
[1]: import pandas as pd
import numpy as np
from matplotlib import pyplot as plt

df = pd.read_excel("/home/aditya/github/Deep-learning-prerequisite/pandas/
↳dataset/weather_data_5_cities.xlsx")
df
```

```
[1]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
0	New York	2025-01-01	14.0	42.7	13.8	
1	New York	2025-01-02	29.4	77.1	12.2	
2	New York	2025-01-03	26.7	69.0	17.5	
3	New York	2025-01-04	30.4	58.6	2.1	
4	New York	2025-01-05	10.4	41.4	1.9	
5	New York	2025-01-06	23.5	72.2	15.7	
6	New York	2025-01-07	23.7	35.5	3.9	
7	New York	2025-01-08	30.8	45.1	2.7	
8	New York	2025-01-09	28.8	37.7	17.4	
9	New York	2025-01-10	31.5	89.6	2.3	
10	London	2025-01-01	13.3	42.9	11.4	
11	London	2025-01-02	0.5	43.1	7.8	
12	London	2025-01-03	7.9	84.0	0.6	
13	London	2025-01-04	11.9	53.0	7.3	
14	London	2025-01-05	0.9	65.2	1.6	
15	London	2025-01-06	0.8	65.7	2.4	
16	London	2025-01-07	13.7	60.8	14.8	
17	London	2025-01-08	10.1	80.1	17.2	
18	London	2025-01-09	30.8	58.4	15.1	
19	London	2025-01-10	9.2	46.8	0.3	
20	Tokyo	2025-01-01	25.1	65.2	10.1	
21	Tokyo	2025-01-02	31.9	42.4	10.6	
22	Tokyo	2025-01-03	10.3	64.2	9.5	
23	Tokyo	2025-01-04	12.2	48.5	5.1	
24	Tokyo	2025-01-05	3.1	32.4	5.7	
25	Tokyo	2025-01-06	21.2	33.1	3.3	

26	Tokyo	2025-01-07	1.1	41.1	12.5
27	Tokyo	2025-01-08	2.9	49.6	7.1
28	Tokyo	2025-01-09	21.8	62.5	2.2
29	Tokyo	2025-01-10	4.6	72.7	16.2
30	Mumbai	2025-01-01	2.2	35.5	0.8
31	Mumbai	2025-01-02	9.0	68.2	17.2
32	Mumbai	2025-01-03	15.9	64.9	18.1
33	Mumbai	2025-01-04	21.5	38.2	17.8
34	Mumbai	2025-01-05	13.1	41.8	16.1
35	Mumbai	2025-01-06	32.7	57.7	16.2
36	Mumbai	2025-01-07	20.8	57.1	3.4
37	Mumbai	2025-01-08	31.0	80.7	5.9
38	Mumbai	2025-01-09	34.3	51.0	17.9
39	Mumbai	2025-01-10	9.3	73.4	13.7
40	Sydney	2025-01-01	26.2	36.4	3.1
41	Sydney	2025-01-02	11.0	74.8	14.0
42	Sydney	2025-01-03	32.1	59.9	6.9
43	Sydney	2025-01-04	20.8	34.0	2.0
44	Sydney	2025-01-05	24.5	42.8	14.3
45	Sydney	2025-01-06	17.5	67.9	18.5
46	Sydney	2025-01-07	13.8	74.5	3.2
47	Sydney	2025-01-08	9.2	77.2	0.1
48	Sydney	2025-01-09	29.0	50.3	14.3
49	Sydney	2025-01-10	23.2	69.1	5.2

	Precipitation_mm
0	12.2
1	11.3
2	9.6
3	12.9
4	12.3
5	7.3
6	11.9
7	3.9
8	6.3
9	5.2
10	14.4
11	2.1
12	3.7
13	3.7
14	10.7
15	11.2
16	6.0
17	2.6
18	7.9
19	9.1
20	14.9

21	2.6
22	5.7
23	8.6
24	10.2
25	4.7
26	9.8
27	11.8
28	6.8
29	9.4
30	13.6
31	0.8
32	4.4
33	4.7
34	0.3
35	8.9
36	0.4
37	11.9
38	9.9
39	12.1
40	7.2
41	12.6
42	11.9
43	4.6
44	5.8
45	10.4
46	11.9
47	5.5
48	6.9
49	3.9

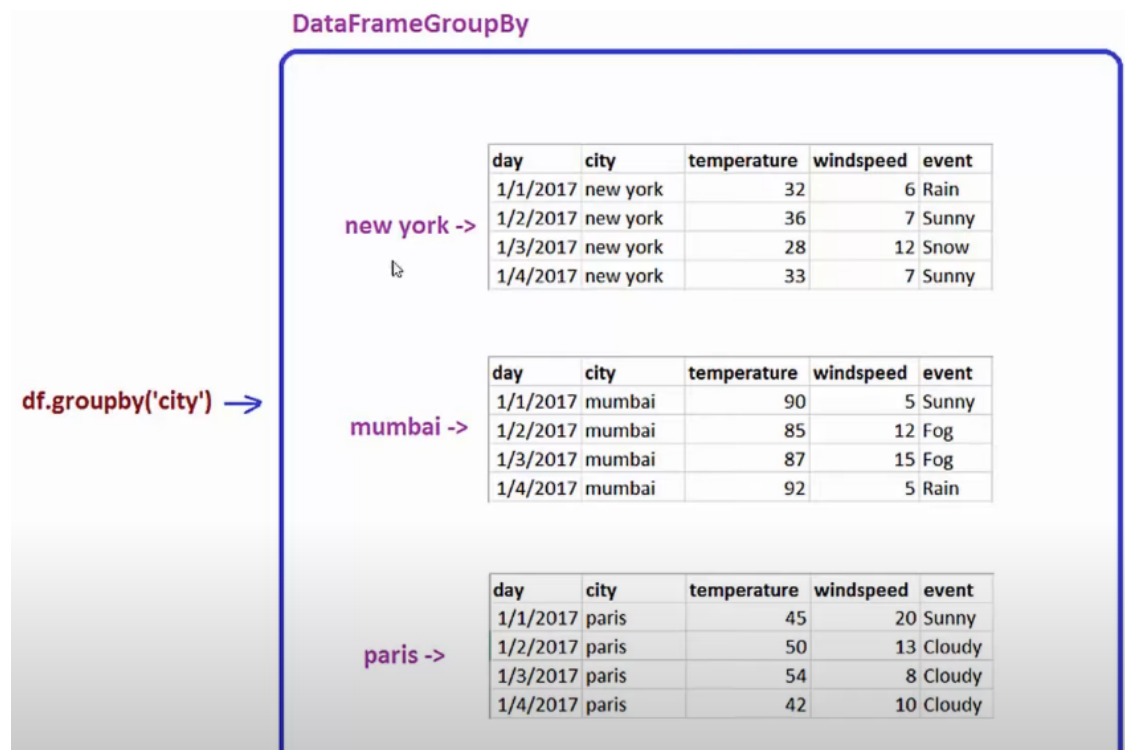
1.1 Find maximum temperature, windspeed, precipitation and humidity in each of the cities.

Here I am trying to group the data per city and then I want to run analytics for each of these groups. For such a usecase it makes sense to use pandas group by feature.

```
[2]: group_by_cities_df = df.groupby("City")
      group_by_cities_df
```

```
[2]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x7efccd497550>
```

1.1.1 So what is inside this group_by pandas object : `<pandas.core.groupby.generic.DataFrameGroupBy object at 0x7fc0c725f100>`



So when you say `dataFrame.groupby(city)` then pandas will create the object as shown in the above figure, where you have keys and values, So here my key is New York and my value is a dataFrame containing Newyork city's data. Same way I have key and data for Mumbai and Paris.

```
[3]: for city, city_df in group_by_cities_df:
      print(city)
      print(city_df)
```

London

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
10	London	2025-01-01	13.3	42.9	11.4	
11	London	2025-01-02	0.5	43.1	7.8	
12	London	2025-01-03	7.9	84.0	0.6	
13	London	2025-01-04	11.9	53.0	7.3	
14	London	2025-01-05	0.9	65.2	1.6	
15	London	2025-01-06	0.8	65.7	2.4	
16	London	2025-01-07	13.7	60.8	14.8	
17	London	2025-01-08	10.1	80.1	17.2	
18	London	2025-01-09	30.8	58.4	15.1	
19	London	2025-01-10	9.2	46.8	0.3	

Precipitation_mm

10	14.4
11	2.1
12	3.7

13	3.7
14	10.7
15	11.2
16	6.0
17	2.6
18	7.9
19	9.1

Mumbai

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
30	Mumbai	2025-01-01	2.2	35.5	0.8	
31	Mumbai	2025-01-02	9.0	68.2	17.2	
32	Mumbai	2025-01-03	15.9	64.9	18.1	
33	Mumbai	2025-01-04	21.5	38.2	17.8	
34	Mumbai	2025-01-05	13.1	41.8	16.1	
35	Mumbai	2025-01-06	32.7	57.7	16.2	
36	Mumbai	2025-01-07	20.8	57.1	3.4	
37	Mumbai	2025-01-08	31.0	80.7	5.9	
38	Mumbai	2025-01-09	34.3	51.0	17.9	
39	Mumbai	2025-01-10	9.3	73.4	13.7	

Precipitation_mm

30	13.6
31	0.8
32	4.4
33	4.7
34	0.3
35	8.9
36	0.4
37	11.9
38	9.9
39	12.1

New York

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
0	New York	2025-01-01	14.0	42.7	13.8	
1	New York	2025-01-02	29.4	77.1	12.2	
2	New York	2025-01-03	26.7	69.0	17.5	
3	New York	2025-01-04	30.4	58.6	2.1	
4	New York	2025-01-05	10.4	41.4	1.9	
5	New York	2025-01-06	23.5	72.2	15.7	
6	New York	2025-01-07	23.7	35.5	3.9	
7	New York	2025-01-08	30.8	45.1	2.7	
8	New York	2025-01-09	28.8	37.7	17.4	
9	New York	2025-01-10	31.5	89.6	2.3	

Precipitation_mm

0	12.2
1	11.3
2	9.6

3	12.9
4	12.3
5	7.3
6	11.9
7	3.9
8	6.3
9	5.2

Sydney

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
40	Sydney	2025-01-01	26.2	36.4	3.1	
41	Sydney	2025-01-02	11.0	74.8	14.0	
42	Sydney	2025-01-03	32.1	59.9	6.9	
43	Sydney	2025-01-04	20.8	34.0	2.0	
44	Sydney	2025-01-05	24.5	42.8	14.3	
45	Sydney	2025-01-06	17.5	67.9	18.5	
46	Sydney	2025-01-07	13.8	74.5	3.2	
47	Sydney	2025-01-08	9.2	77.2	0.1	
48	Sydney	2025-01-09	29.0	50.3	14.3	
49	Sydney	2025-01-10	23.2	69.1	5.2	

Precipitation_mm

40	7.2
41	12.6
42	11.9
43	4.6
44	5.8
45	10.4
46	11.9
47	5.5
48	6.9
49	3.9

Tokyo

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
20	Tokyo	2025-01-01	25.1	65.2	10.1	
21	Tokyo	2025-01-02	31.9	42.4	10.6	
22	Tokyo	2025-01-03	10.3	64.2	9.5	
23	Tokyo	2025-01-04	12.2	48.5	5.1	
24	Tokyo	2025-01-05	3.1	32.4	5.7	
25	Tokyo	2025-01-06	21.2	33.1	3.3	
26	Tokyo	2025-01-07	1.1	41.1	12.5	
27	Tokyo	2025-01-08	2.9	49.6	7.1	
28	Tokyo	2025-01-09	21.8	62.5	2.2	
29	Tokyo	2025-01-10	4.6	72.7	16.2	

Precipitation_mm

20	14.9
21	2.6
22	5.7

23	8.6
24	10.2
25	4.7
26	9.8
27	11.8
28	6.8
29	9.4

1.1.2 If you want to access a specific dataFrame of a particular city then you can do something like this

- `group_by_cities_df.get_group("Mumbai")` As you know the `group_by_cities_df` is an object created by pandas where city names are the keys and their values are the dataFrame containing the weather data of those respective cities. By providing the key i.e city name here in this case you will be able to access the dataFrame associated with that city.
- This statement in pandas `group_by_cities_df.get_group("Mumbai")` is similar to this SQL statement `SELECT * FROM city_data GROUP BY city`. This SQL statement is gonna give you similar result, where it will group the data by the cities.

Access the dataFrame of Mumbai city

```
[4]: mumbai_df = group_by_cities_df.get_group("Mumbai")
mumbai_df
```

```
[4]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
30	Mumbai	2025-01-01	2.2	35.5	0.8	
31	Mumbai	2025-01-02	9.0	68.2	17.2	
32	Mumbai	2025-01-03	15.9	64.9	18.1	
33	Mumbai	2025-01-04	21.5	38.2	17.8	
34	Mumbai	2025-01-05	13.1	41.8	16.1	
35	Mumbai	2025-01-06	32.7	57.7	16.2	
36	Mumbai	2025-01-07	20.8	57.1	3.4	
37	Mumbai	2025-01-08	31.0	80.7	5.9	
38	Mumbai	2025-01-09	34.3	51.0	17.9	
39	Mumbai	2025-01-10	9.3	73.4	13.7	

	Precipitation_mm
30	13.6
31	0.8
32	4.4
33	4.7
34	0.3
35	8.9
36	0.4
37	11.9
38	9.9
39	12.1

Get the maximum temperature , humidity windspeed and precipitation in mumbai city.

```
[5]: mumbai_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪max()
```

```
[5]: Temperature_C      34.3
      Humidity_%        80.7
      WindSpeed_kmph    18.1
      Precipitation_mm  13.6
      dtype: float64
```

Access the dataframe of New York city

```
[6]: newyork_df = group_by_cities_df.get_group("New York")
      newyork_df
```

```
[6]:      City      Date  Temperature_C  Humidity_%  WindSpeed_kmph  \
0  New York 2025-01-01           14.0         42.7           13.8
1  New York 2025-01-02           29.4         77.1           12.2
2  New York 2025-01-03           26.7         69.0           17.5
3  New York 2025-01-04           30.4         58.6            2.1
4  New York 2025-01-05           10.4         41.4            1.9
5  New York 2025-01-06           23.5         72.2           15.7
6  New York 2025-01-07           23.7         35.5            3.9
7  New York 2025-01-08           30.8         45.1            2.7
8  New York 2025-01-09           28.8         37.7           17.4
9  New York 2025-01-10           31.5         89.6            2.3

      Precipitation_mm
0              12.2
1              11.3
2               9.6
3              12.9
4              12.3
5               7.3
6              11.9
7               3.9
8               6.3
9               5.2
```

Get the maximum temperature, humidity , windspeed and precipitation in new york city.

```
[7]: newyork_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph",
      ↪"Precipitation_mm"]].max()
```

```
[7]: Temperature_C      31.5
      Humidity_%        89.6
      WindSpeed_kmph    17.5
```



```
Precipitation_mm    12.9
dtype: float64
```

Access the dataframe of Tokyo city

```
[8]: tokyo_df = group_by_cities_df.get_group("Tokyo")
tokyo_df
```

```
[8]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
20	Tokyo	2025-01-01	25.1	65.2	10.1	
21	Tokyo	2025-01-02	31.9	42.4	10.6	
22	Tokyo	2025-01-03	10.3	64.2	9.5	
23	Tokyo	2025-01-04	12.2	48.5	5.1	
24	Tokyo	2025-01-05	3.1	32.4	5.7	
25	Tokyo	2025-01-06	21.2	33.1	3.3	
26	Tokyo	2025-01-07	1.1	41.1	12.5	
27	Tokyo	2025-01-08	2.9	49.6	7.1	
28	Tokyo	2025-01-09	21.8	62.5	2.2	
29	Tokyo	2025-01-10	4.6	72.7	16.2	

	Precipitation_mm
20	14.9
21	2.6
22	5.7
23	8.6
24	10.2
25	4.7
26	9.8
27	11.8
28	6.8
29	9.4

Get the maximum temperature, humidity , windspeed and precipitation in tokyo city.

```
[9]: tokyo_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
     ↪max()
```

```
[9]: Temperature_C    31.9
Humidity_%          72.7
WindSpeed_kmph      16.2
Precipitation_mm    14.9
dtype: float64
```

Access the dataframe of Sydney city

```
[10]: sydney_df = group_by_cities_df.get_group("Sydney")
sydney_df
```

```
[10]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
40	Sydney	2025-01-01	26.2	36.4	3.1	
41	Sydney	2025-01-02	11.0	74.8	14.0	
42	Sydney	2025-01-03	32.1	59.9	6.9	
43	Sydney	2025-01-04	20.8	34.0	2.0	
44	Sydney	2025-01-05	24.5	42.8	14.3	
45	Sydney	2025-01-06	17.5	67.9	18.5	
46	Sydney	2025-01-07	13.8	74.5	3.2	
47	Sydney	2025-01-08	9.2	77.2	0.1	
48	Sydney	2025-01-09	29.0	50.3	14.3	
49	Sydney	2025-01-10	23.2	69.1	5.2	

	Precipitation_mm
40	7.2
41	12.6
42	11.9
43	4.6
44	5.8
45	10.4
46	11.9
47	5.5
48	6.9
49	3.9

Get the maximum temperature, humidity, windspeed and precipitation in sydney city

```
[11]: sydney_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪max()
```

```
[11]: Temperature_C      32.1
      Humidity_%        77.2
      WindSpeed_kmph    18.5
      Precipitation_mm   12.6
      dtype: float64
```

Access the dataframe of London city

```
[12]: london_df = group_by_cities_df.get_group("London")
      london_df
```

```
[12]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
10	London	2025-01-01	13.3	42.9	11.4	
11	London	2025-01-02	0.5	43.1	7.8	
12	London	2025-01-03	7.9	84.0	0.6	
13	London	2025-01-04	11.9	53.0	7.3	
14	London	2025-01-05	0.9	65.2	1.6	
15	London	2025-01-06	0.8	65.7	2.4	
16	London	2025-01-07	13.7	60.8	14.8	
17	London	2025-01-08	10.1	80.1	17.2	

18	London	2025-01-09	30.8	58.4	15.1
19	London	2025-01-10	9.2	46.8	0.3

	Precipitation_mm
10	14.4
11	2.1
12	3.7
13	3.7
14	10.7
15	11.2
16	6.0
17	2.6
18	7.9
19	9.1

Get the maximum temperature, humidity, windspeed and precipitation in london city

```
[13]: london_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪max()
```

```
[13]: Temperature_C      30.8
      Humidity_%        84.0
      WindSpeed_kmph    17.2
      Precipitation_mm   14.4
      dtype: float64
```

1.2 Find minimum temperature , humidity windspeed and precipitation in each of the cities.

Get the minimum temperature, humidity, precipitation and windspeed of Mumbai city

```
[14]: mumbai_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪min()
```

```
[14]: Temperature_C      2.2
      Humidity_%        35.5
      WindSpeed_kmph     0.8
      Precipitation_mm    0.3
      dtype: float64
```

Get the minimum temperature, humidity, precipitation and windspeed of New York city

```
[15]: newyork_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪min()
```

```
[15]: Temperature_C      10.4
      Humidity_%        35.5
      WindSpeed_kmph     1.9
```

```
Precipitation_mm      3.9
dtype: float64
```

Get the minimum temperature , humidity , windspeed and precipitation of London city

```
[16]: london_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪min()
```

```
[16]: Temperature_C      0.5
      Humidity_%       42.9
      WindSpeed_kmph    0.3
      Precipitation_mm  2.1
      dtype: float64
```

Get the minimum temperature, humidity, windspeed and precipitation of Tokyo city

```
[17]: tokyo_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪min()
```

```
[17]: Temperature_C      1.1
      Humidity_%       32.4
      WindSpeed_kmph    2.2
      Precipitation_mm  2.6
      dtype: float64
```

Get the minimum temperature, humidity, windspeed and precipitation of sydney city

```
[18]: sydney_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪min()
```

```
[18]: Temperature_C      9.2
      Humidity_%       34.0
      WindSpeed_kmph    0.1
      Precipitation_mm  3.9
      dtype: float64
```

1.3 Find average (mean) temperature, windspeed, humidity and precipitation per city

Get the average (mean) of temperature, humidity, precipitation and windspeed of Mumbai city

```
[19]: mumbai_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪mean()
```

```
[19]: Temperature_C      18.98
      Humidity_%       56.85
      WindSpeed_kmph   12.71
      Precipitation_mm  6.70
      dtype: float64
```

Get the average (mean) of temperature, windspeed, humidity and precipitation of Tokyo city

```
[20]: tokyo_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪mean()
```

```
[20]: Temperature_C      13.42  
      Humidity_%        51.17  
      WindSpeed_kmph    8.23  
      Precipitation_mm  8.45  
      dtype: float64
```

Get the average (mean) of temperature, humidity , windspeed and precipitation of Sydney city

```
[21]: sydney_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪mean()
```

```
[21]: Temperature_C      20.73  
      Humidity_%        58.69  
      WindSpeed_kmph    8.16  
      Precipitation_mm  8.07  
      dtype: float64
```

Get the average (mean) of temperature , humidity , windspeed and precipitation of New York city

```
[22]: newyork_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪mean()
```

```
[22]: Temperature_C      24.92  
      Humidity_%        56.89  
      WindSpeed_kmph    8.95  
      Precipitation_mm  9.29  
      dtype: float64
```

Get the average (mean) of temperature, humidity , windspeed and precipitation of London city

```
[23]: london_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪mean()
```

```
[23]: Temperature_C      9.91  
      Humidity_%        60.00  
      WindSpeed_kmph    7.85  
      Precipitation_mm  7.14  
      dtype: float64
```

1.4 Find the median of temperature, humidity, windspeed and precipitation of each cities

Get the median of temperature , humidity , windspeed and precipitation of Mumbai city

```
[24]: mumbai_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪median()
```

```
[24]: Temperature_C      18.35  
      Humidity_%        57.40  
      WindSpeed_kmph    16.15  
      Precipitation_mm   6.80  
      dtype: float64
```

Get the median of temperature , humidity , windspeed and precipitation of Tokyo city

```
[25]: tokyo_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪median()
```

```
[25]: Temperature_C      11.25  
      Humidity_%        49.05  
      WindSpeed_kmph     8.30  
      Precipitation_mm   9.00  
      dtype: float64
```

Get the median of temperature, humidity, windspeed and precipitation of Sydney city

```
[26]: sydney_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪median()
```

```
[26]: Temperature_C      22.00  
      Humidity_%        63.90  
      WindSpeed_kmph     6.05  
      Precipitation_mm   7.05  
      dtype: float64
```

Get the median of temperature, humidity, windspeed and precipitation of London city

```
[27]: london_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪median()
```

```
[27]: Temperature_C      9.65  
      Humidity_%        59.60  
      WindSpeed_kmph     7.55  
      Precipitation_mm   6.95  
      dtype: float64
```

Get the median of temperature, humidity, windspeed and precipitation of New York city

```
[28]: newyork_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪median()
```

```
[28]: Temperature_C      27.75
      Humidity_%        51.85
      WindSpeed_kmph    8.05
      Precipitation_mm  10.45
      dtype: float64
```

1.5 Find the mode of temperature, humidity, windspeed and precipitation of each of the cities

Get the mode of temperature , humidity , precipitation and windspeed of Mumbai city

```
[29]: mumbai_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪mode()
```

```
[29]:   Temperature_C  Humidity_%  WindSpeed_kmph  Precipitation_mm
0             2.2         35.5             0.8             0.3
1             9.0         38.2             3.4             0.4
2             9.3         41.8             5.9             0.8
3            13.1         51.0            13.7             4.4
4            15.9         57.1            16.1             4.7
5            20.8         57.7            16.2             8.9
6            21.5         64.9            17.2             9.9
7            31.0         68.2            17.8            11.9
8            32.7         73.4            17.9            12.1
9            34.3         80.7            18.1            13.6
```

Get the mode of temperature, humidity, precipitation and windspeed of Tokyo city

```
[30]: tokyo_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪mode()
```

```
[30]:   Temperature_C  Humidity_%  WindSpeed_kmph  Precipitation_mm
0             1.1         32.4             2.2             2.6
1             2.9         33.1             3.3             4.7
2             3.1         41.1             5.1             5.7
3             4.6         42.4             5.7             6.8
4            10.3         48.5             7.1             8.6
5            12.2         49.6             9.5             9.4
6            21.2         62.5            10.1             9.8
7            21.8         64.2            10.6            10.2
8            25.1         65.2            12.5            11.8
9            31.9         72.7            16.2            14.9
```

Get the mode of temperature, humidity , windspeed and precipitation of Sydney city

```
[31]: sydney_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪mode()
```

```
[31]:   Temperature_C  Humidity_%  WindSpeed_kmph  Precipitation_mm
0           9.2        34.0         14.3         11.9
1          11.0        36.4          NaN          NaN
2          13.8        42.8          NaN          NaN
3          17.5        50.3          NaN          NaN
4          20.8        59.9          NaN          NaN
5          23.2        67.9          NaN          NaN
6          24.5        69.1          NaN          NaN
7          26.2        74.5          NaN          NaN
8          29.0        74.8          NaN          NaN
9          32.1        77.2          NaN          NaN
```

Get the mode of temperature, humidity, windspeed and precipitation of New York city

```
[32]: newyork_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪mode()
```

```
[32]:   Temperature_C  Humidity_%  WindSpeed_kmph  Precipitation_mm
0           10.4        35.5          1.9          3.9
1           14.0        37.7          2.1          5.2
2           23.5        41.4          2.3          6.3
3           23.7        42.7          2.7          7.3
4           26.7        45.1          3.9          9.6
5           28.8        58.6         12.2         11.3
6           29.4        69.0         13.8         11.9
7           30.4        72.2         15.7         12.2
8           30.8        77.1         17.4         12.3
9           31.5        89.6         17.5         12.9
```

Get the mode of temperature, humidity, windspeed and Precipitation of London city

```
[33]: london_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].
      ↪mode()
```

```
[33]:   Temperature_C  Humidity_%  WindSpeed_kmph  Precipitation_mm
0           0.5        42.9          0.3          3.7
1           0.8        43.1          0.6          NaN
2           0.9        46.8          1.6          NaN
3           7.9        53.0          2.4          NaN
4           9.2        58.4          7.3          NaN
5          10.1        60.8          7.8          NaN
6          11.9        65.2         11.4          NaN
7          13.3        65.7         14.8          NaN
8          13.7        80.1         15.1          NaN
9          30.8        84.0         17.2          NaN
```


1.6 Find the standard deviation of temperature, humidity, windspeed and precipitation of each of the cities

```
[70]: mumbai_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪std()
```

```
[70]: Temperature_C      11.035377  
      Humidity_%        15.280725  
      WindSpeed_kmph     6.680062  
      Precipitation_mm   5.202991  
      dtype: float64
```

```
[71]: sydney_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪std()
```

```
[71]: Temperature_C      7.702676  
      Humidity_%        16.586772  
      WindSpeed_kmph     6.496871  
      Precipitation_mm   3.309935  
      dtype: float64
```

```
[72]: london_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪std()
```

```
[72]: Temperature_C      8.966041  
      Humidity_%        14.313940  
      WindSpeed_kmph     6.493800  
      Precipitation_mm   4.180165  
      dtype: float64
```

```
[73]: newyork_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪std()
```

```
[73]: Temperature_C      7.301263  
      Humidity_%        19.056784  
      WindSpeed_kmph     6.908650  
      Precipitation_mm   3.336482  
      dtype: float64
```

```
[74]: tokyo_df[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].  
      ↪std()
```

```
[74]: Temperature_C      10.886567  
      Humidity_%        14.241102  
      WindSpeed_kmph     4.354066  
      Precipitation_mm   3.608709  
      dtype: float64
```

1.7 Lets see how to calculate min(), max(), mean(), mode(), median() and show the results of all the cities in a single dataframe

Here I will be creating a new dataframe for this demo

```
[34]: df2 = pd.read_excel("/home/aditya/github/Deep-learning-prerequisite/pandas/  
↳dataset/weather_data_5_cities.xlsx")  
df2
```

```
[34]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
0	New York	2025-01-01	14.0	42.7	13.8	
1	New York	2025-01-02	29.4	77.1	12.2	
2	New York	2025-01-03	26.7	69.0	17.5	
3	New York	2025-01-04	30.4	58.6	2.1	
4	New York	2025-01-05	10.4	41.4	1.9	
5	New York	2025-01-06	23.5	72.2	15.7	
6	New York	2025-01-07	23.7	35.5	3.9	
7	New York	2025-01-08	30.8	45.1	2.7	
8	New York	2025-01-09	28.8	37.7	17.4	
9	New York	2025-01-10	31.5	89.6	2.3	
10	London	2025-01-01	13.3	42.9	11.4	
11	London	2025-01-02	0.5	43.1	7.8	
12	London	2025-01-03	7.9	84.0	0.6	
13	London	2025-01-04	11.9	53.0	7.3	
14	London	2025-01-05	0.9	65.2	1.6	
15	London	2025-01-06	0.8	65.7	2.4	
16	London	2025-01-07	13.7	60.8	14.8	
17	London	2025-01-08	10.1	80.1	17.2	
18	London	2025-01-09	30.8	58.4	15.1	
19	London	2025-01-10	9.2	46.8	0.3	
20	Tokyo	2025-01-01	25.1	65.2	10.1	
21	Tokyo	2025-01-02	31.9	42.4	10.6	
22	Tokyo	2025-01-03	10.3	64.2	9.5	
23	Tokyo	2025-01-04	12.2	48.5	5.1	
24	Tokyo	2025-01-05	3.1	32.4	5.7	
25	Tokyo	2025-01-06	21.2	33.1	3.3	
26	Tokyo	2025-01-07	1.1	41.1	12.5	
27	Tokyo	2025-01-08	2.9	49.6	7.1	
28	Tokyo	2025-01-09	21.8	62.5	2.2	
29	Tokyo	2025-01-10	4.6	72.7	16.2	
30	Mumbai	2025-01-01	2.2	35.5	0.8	
31	Mumbai	2025-01-02	9.0	68.2	17.2	
32	Mumbai	2025-01-03	15.9	64.9	18.1	
33	Mumbai	2025-01-04	21.5	38.2	17.8	
34	Mumbai	2025-01-05	13.1	41.8	16.1	
35	Mumbai	2025-01-06	32.7	57.7	16.2	
36	Mumbai	2025-01-07	20.8	57.1	3.4	
37	Mumbai	2025-01-08	31.0	80.7	5.9	
38	Mumbai	2025-01-09	34.3	51.0	17.9	

39	Mumbai	2025-01-10	9.3	73.4	13.7
40	Sydney	2025-01-01	26.2	36.4	3.1
41	Sydney	2025-01-02	11.0	74.8	14.0
42	Sydney	2025-01-03	32.1	59.9	6.9
43	Sydney	2025-01-04	20.8	34.0	2.0
44	Sydney	2025-01-05	24.5	42.8	14.3
45	Sydney	2025-01-06	17.5	67.9	18.5
46	Sydney	2025-01-07	13.8	74.5	3.2
47	Sydney	2025-01-08	9.2	77.2	0.1
48	Sydney	2025-01-09	29.0	50.3	14.3
49	Sydney	2025-01-10	23.2	69.1	5.2

	Precipitation_mm
0	12.2
1	11.3
2	9.6
3	12.9
4	12.3
5	7.3
6	11.9
7	3.9
8	6.3
9	5.2
10	14.4
11	2.1
12	3.7
13	3.7
14	10.7
15	11.2
16	6.0
17	2.6
18	7.9
19	9.1
20	14.9
21	2.6
22	5.7
23	8.6
24	10.2
25	4.7
26	9.8
27	11.8
28	6.8
29	9.4
30	13.6
31	0.8
32	4.4
33	4.7

```

34          0.3
35          8.9
36          0.4
37         11.9
38          9.9
39         12.1
40          7.2
41         12.6
42         11.9
43          4.6
44          5.8
45         10.4
46         11.9
47          5.5
48          6.9
49          3.9

```

```
[35]: group_by_df2 = df2.groupby("City")
      group_by_df2
```

```
[35]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x7efc93feb700>
```

1.7.1 Get the max temperature, humidity, windspeed and precipitation of each of the cities in a single dataFrame:

```
[36]: group_by_df2.max()
```

```
[36]:
```

	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
City					
London	2025-01-10	30.8	84.0	17.2	
Mumbai	2025-01-10	34.3	80.7	18.1	
New York	2025-01-10	31.5	89.6	17.5	
Sydney	2025-01-10	32.1	77.2	18.5	
Tokyo	2025-01-10	31.9	72.7	16.2	

	Precipitation_mm
City	
London	14.4
Mumbai	13.6
New York	12.9
Sydney	12.6
Tokyo	14.9

1.7.2 Get the min temperature, humidity, windspeed and precipitation of each of the cities in a single dataFrame:

```
[37]: group_by_df2.min()
```

```
[37]:
```

	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
City					
London	2025-01-01	0.5	42.9	0.3	
Mumbai	2025-01-01	2.2	35.5	0.8	
New York	2025-01-01	10.4	35.5	1.9	
Sydney	2025-01-01	9.2	34.0	0.1	
Tokyo	2025-01-01	1.1	32.4	2.2	

	Precipitation_mm
City	
London	2.1
Mumbai	0.3
New York	3.9
Sydney	3.9
Tokyo	2.6

1.7.3 Get the mean of temperature, humidity, windspeed and precipitation of each of the cities in a single dataFrame:

```
[38]: group_by_df2.mean()
```

```
[38]:
```

	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
City					
London	2025-01-05 12:00:00	9.91	60.00	7.85	
Mumbai	2025-01-05 12:00:00	18.98	56.85	12.71	
New York	2025-01-05 12:00:00	24.92	56.89	8.95	
Sydney	2025-01-05 12:00:00	20.73	58.69	8.16	
Tokyo	2025-01-05 12:00:00	13.42	51.17	8.23	

	Precipitation_mm
City	
London	7.14
Mumbai	6.70
New York	9.29
Sydney	8.07
Tokyo	8.45

1.7.4 Get the median of temperature, humidity, windspeed and precipitation of each of the cities in a single dataFrame:

```
[39]: group_by_df2.median()
```

```
[39]:
```

	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
City					
London	2025-01-05 12:00:00	9.65	59.60	7.55	
Mumbai	2025-01-05 12:00:00	18.35	57.40	16.15	
New York	2025-01-05 12:00:00	27.75	51.85	8.05	
Sydney	2025-01-05 12:00:00	22.00	63.90	6.05	
Tokyo	2025-01-05 12:00:00	11.25	49.05	8.30	

	Precipitation_mm
City	
London	6.95
Mumbai	6.80
New York	10.45
Sydney	7.05
Tokyo	9.00

1.7.5 Get the mode of temperature, humidity, windspeed and precipitation of each cities in a single DataFrame:

```
[40]: group_by_df2.mode()
```

```
-----
AttributeError                                Traceback (most recent call last)
Cell In[40], line 1
----> 1 group_by_df2.mode()

File ~/miniconda3/envs/tensorflow/lib/python3.10/site-packages/pandas/core/
↳groupby/groupby.py:1363, in GroupBy.__getattr__(self, attr)
    1360 if attr in self.obj:
    1361     return self[attr]
-> 1363 raise AttributeError(
    1364     f'"{type(self).__name__}" object has no attribute "{attr}"'
    1365 )

AttributeError: 'DataFrameGroupBy' object has no attribute 'mode'
```

1.7.6 Explanation

What does this error mean?

AttributeError: 'DataFrameGroupBy' object has no attribute 'mode'

- This error means that : in Pandas —>
 - Hey you are trying to call `.mode()` directly on a `GroupBy` object, but I (Pandas) don't know how to do that for a group object without you telling me what to do with each group.

Why does this error happen ?

- `group_by_df2` is a `DataFrameGroupBy` object, essentially a “Lazy” object that only knows:
 - How to split your data by “City”
 - How to apply certain functions (like `.mean()`, `.sum()`, `.max()`) to each group
- Some aggregation method like `.max()` and `.mean()` are implemented natively for `GroupBy` objects but `.mode()` is not implemented in the same direct way for `GroupBy`.
- `.mode()` in pandas is actually a `DataFrame` method, not a `GroupBy` aggregation. The `GroupBy` engine doesn’t just pass `.mode()` to each subgroup automatically.
- This is why :

```
df.groupby('City').max() # This works
df.groupby('City').mode() # This doesn't work
```

solution :

```
[66]: group_by_df2.apply(lambda x :  
    ↪x[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].mode())
```

/tmp/ipykernel_12137/2882096768.py:1: DeprecationWarning: `DataFrameGroupBy.apply` operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass ``include_groups=False`` to exclude the groupings or explicitly select the grouping columns after `groupby` to silence this warning.

```
group_by_df2.apply(lambda x :  
x[["Temperature_C", "Humidity_%", "WindSpeed_kmph", "Precipitation_mm"]].mode())
```

```
[66]:
```

		Temperature_C	Humidity_%	WindSpeed_kmph	Precipitation_mm
City					
London	0	0.5	42.9	0.3	3.7
	1	0.8	43.1	0.6	NaN
	2	0.9	46.8	1.6	NaN
	3	7.9	53.0	2.4	NaN
	4	9.2	58.4	7.3	NaN
	5	10.1	60.8	7.8	NaN
	6	11.9	65.2	11.4	NaN
	7	13.3	65.7	14.8	NaN
	8	13.7	80.1	15.1	NaN
Mumbai	9	30.8	84.0	17.2	NaN
	0	2.2	35.5	0.8	0.3
	1	9.0	38.2	3.4	0.4
	2	9.3	41.8	5.9	0.8
	3	13.1	51.0	13.7	4.4
	4	15.9	57.1	16.1	4.7
	5	20.8	57.7	16.2	8.9
	6	21.5	64.9	17.2	9.9
	7	31.0	68.2	17.8	11.9
	8	32.7	73.4	17.9	12.1

	9	34.3	80.7	18.1	13.6
New York	0	10.4	35.5	1.9	3.9
	1	14.0	37.7	2.1	5.2
	2	23.5	41.4	2.3	6.3
	3	23.7	42.7	2.7	7.3
	4	26.7	45.1	3.9	9.6
	5	28.8	58.6	12.2	11.3
	6	29.4	69.0	13.8	11.9
	7	30.4	72.2	15.7	12.2
	8	30.8	77.1	17.4	12.3
	9	31.5	89.6	17.5	12.9
Sydney	0	9.2	34.0	14.3	11.9
	1	11.0	36.4	NaN	NaN
	2	13.8	42.8	NaN	NaN
	3	17.5	50.3	NaN	NaN
	4	20.8	59.9	NaN	NaN
	5	23.2	67.9	NaN	NaN
	6	24.5	69.1	NaN	NaN
	7	26.2	74.5	NaN	NaN
	8	29.0	74.8	NaN	NaN
	9	32.1	77.2	NaN	NaN
Tokyo	0	1.1	32.4	2.2	2.6
	1	2.9	33.1	3.3	4.7
	2	3.1	41.1	5.1	5.7
	3	4.6	42.4	5.7	6.8
	4	10.3	48.5	7.1	8.6
	5	12.2	49.6	9.5	9.4
	6	21.2	62.5	10.1	9.8
	7	21.8	64.2	10.6	10.2
	8	25.1	65.2	12.5	11.8
	9	31.9	72.7	16.2	14.9

What is lambda here?

- `lambda` is an anonymous function in python, a function without a name used here as a quick inline definition to select the columns from the `group_by_df2` object of each city's `dataFrame` and calculate mode on it.
- `lambda g: g[['temperature', 'humidity', 'windspeed', 'precipitation']].mode()` is equivalent to writing this

```
def some_function(g):
    return g[['temperature', 'humidity', 'windspeed', 'precipitation']].mode()
```
- You don't need to define a separate `def` function when the logic you want to apply is very short and simple in nature you can just use `lambda` function instead.
- Good for quick one-off logic in `.apply()`

When not to use a lambda function :

- If the logic is long or used multiple times, better to define a named function for readability.
- If debugging, named functions give clearer stack traces.

What is this warning ?

```
/tmp/ipykernel_3514967/2882096768.py:1: DeprecationWarning: DataFrameGroupBy.apply operated on
group_by_df2.apply(lambda x : x[["Temperature_C","Humidity_%","WindSpeed_kmph","Precipitation_mm"]])
```

This warning is Pandas telling you: “Right now, when `.apply()` runs on a grouped DataFrame, it includes the grouping columns (city in my case) in the `x` DataFrame passed to my lambda function. In the future that default will change grouping columns won’t be included unless I explicitly ask”

```
group_by_df2 = df2.groupby("City")
group_by_df2.apply(lambda x: x[["Temperature_C","Humidity_%","WindSpeed_kmph","Precipitation_mm"]])
```

- `x` currently contains “City” plus the four columns you’re selecting.
- In the future, “City” will not be there unless you explicitly keep it.
- Pandas is warning you so you don’t get unexpected behavior after the change.

Solution Select columns before `.apply()`

```
[41]: group_by_df2[["Temperature_C","Humidity_%","WindSpeed_kmph","Precipitation_mm"]].
      ↪ apply(lambda x:x.mode())
```

```
[41]:
```

		Temperature_C	Humidity_%	WindSpeed_kmph	Precipitation_mm
City					
London	0	0.5	42.9	0.3	3.7
	1	0.8	43.1	0.6	NaN
	2	0.9	46.8	1.6	NaN
	3	7.9	53.0	2.4	NaN
	4	9.2	58.4	7.3	NaN
	5	10.1	60.8	7.8	NaN
	6	11.9	65.2	11.4	NaN
	7	13.3	65.7	14.8	NaN
	8	13.7	80.1	15.1	NaN
Mumbai	9	30.8	84.0	17.2	NaN
	0	2.2	35.5	0.8	0.3
	1	9.0	38.2	3.4	0.4
	2	9.3	41.8	5.9	0.8
	3	13.1	51.0	13.7	4.4
	4	15.9	57.1	16.1	4.7
	5	20.8	57.7	16.2	8.9
	6	21.5	64.9	17.2	9.9
	7	31.0	68.2	17.8	11.9
New York	8	32.7	73.4	17.9	12.1
	9	34.3	80.7	18.1	13.6
	0	10.4	35.5	1.9	3.9
	1	14.0	37.7	2.1	5.2

	2	23.5	41.4	2.3	6.3
	3	23.7	42.7	2.7	7.3
	4	26.7	45.1	3.9	9.6
	5	28.8	58.6	12.2	11.3
	6	29.4	69.0	13.8	11.9
	7	30.4	72.2	15.7	12.2
	8	30.8	77.1	17.4	12.3
	9	31.5	89.6	17.5	12.9
Sydney	0	9.2	34.0	14.3	11.9
	1	11.0	36.4	NaN	NaN
	2	13.8	42.8	NaN	NaN
	3	17.5	50.3	NaN	NaN
	4	20.8	59.9	NaN	NaN
	5	23.2	67.9	NaN	NaN
	6	24.5	69.1	NaN	NaN
	7	26.2	74.5	NaN	NaN
	8	29.0	74.8	NaN	NaN
	9	32.1	77.2	NaN	NaN
Tokyo	0	1.1	32.4	2.2	2.6
	1	2.9	33.1	3.3	4.7
	2	3.1	41.1	5.1	5.7
	3	4.6	42.4	5.7	6.8
	4	10.3	48.5	7.1	8.6
	5	12.2	49.6	9.5	9.4
	6	21.2	62.5	10.1	9.8
	7	21.8	64.2	10.6	10.2
	8	25.1	65.2	12.5	11.8
	9	31.9	72.7	16.2	14.9

1.7.7 Get the standard deviation of temperature, humidity, windspeed and precipitation of each cities in a single dataframe:

```
[69]: group_by_df2.std()
```

```
[69]:
```

	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
City					
London	3 days 00:39:48.990594023	8.966041	14.313940	6.493800	
Mumbai	3 days 00:39:48.990594023	11.035377	15.280725	6.680062	
New York	3 days 00:39:48.990594023	7.301263	19.056784	6.908650	
Sydney	3 days 00:39:48.990594023	7.702676	16.586772	6.496871	
Tokyo	3 days 00:39:48.990594023	10.886567	14.241102	4.354066	

	Precipitation_mm
City	
London	4.180165
Mumbai	5.202991
New York	3.336482

Sydney	3.309935
Tokyo	3.608709

1.8 If you want to get all the analytics in one shot then you can also use describe function with the group by object of the dataframe

```
[42]: group_by_df2.describe()
```

```
[42]:
```

	Date							
	count		mean		min		25%	
City								
London	10	2025-01-05 12:00:00	2025-01-01 00:00:00	2025-01-03 06:00:00				
Mumbai	10	2025-01-05 12:00:00	2025-01-01 00:00:00	2025-01-03 06:00:00				
New York	10	2025-01-05 12:00:00	2025-01-01 00:00:00	2025-01-03 06:00:00				
Sydney	10	2025-01-05 12:00:00	2025-01-01 00:00:00	2025-01-03 06:00:00				
Tokyo	10	2025-01-05 12:00:00	2025-01-01 00:00:00	2025-01-03 06:00:00				

			50%		75%		max	std
City								
London	2025-01-05 12:00:00	2025-01-07 18:00:00	2025-01-10 00:00:00	NaN				
Mumbai	2025-01-05 12:00:00	2025-01-07 18:00:00	2025-01-10 00:00:00	NaN				
New York	2025-01-05 12:00:00	2025-01-07 18:00:00	2025-01-10 00:00:00	NaN				
Sydney	2025-01-05 12:00:00	2025-01-07 18:00:00	2025-01-10 00:00:00	NaN				
Tokyo	2025-01-05 12:00:00	2025-01-07 18:00:00	2025-01-10 00:00:00	NaN				

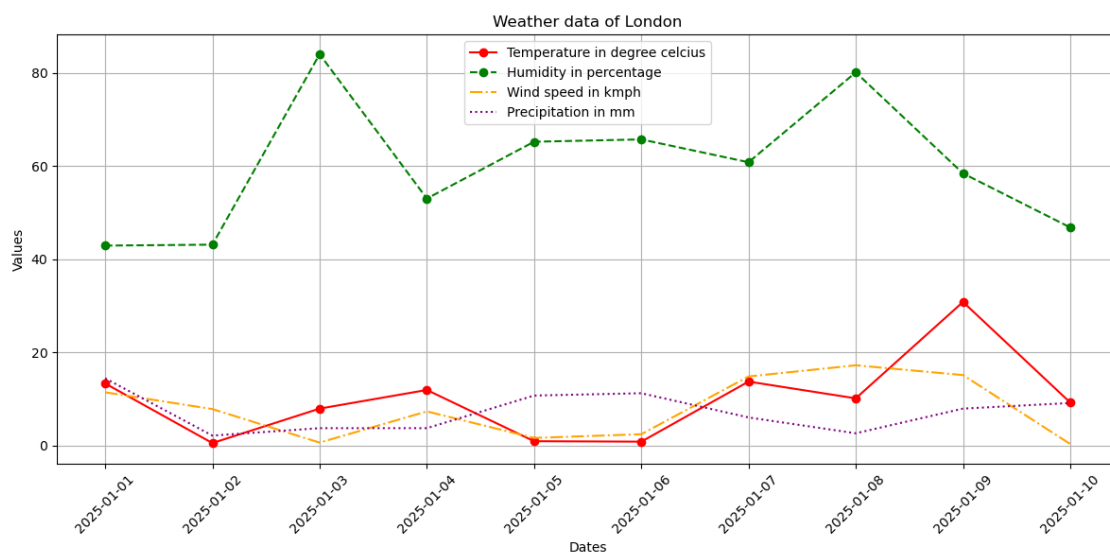
	Temperature_C	...	WindSpeed_kmph		Precipitation_mm	
	count	mean	...	max	std	count
City			...			
London	10.0	9.91	...	17.2	6.493800	10.0
Mumbai	10.0	18.98	...	18.1	6.680062	10.0
New York	10.0	24.92	...	17.5	6.908650	10.0
Sydney	10.0	20.73	...	18.5	6.496871	10.0
Tokyo	10.0	13.42	...	16.2	4.354066	10.0

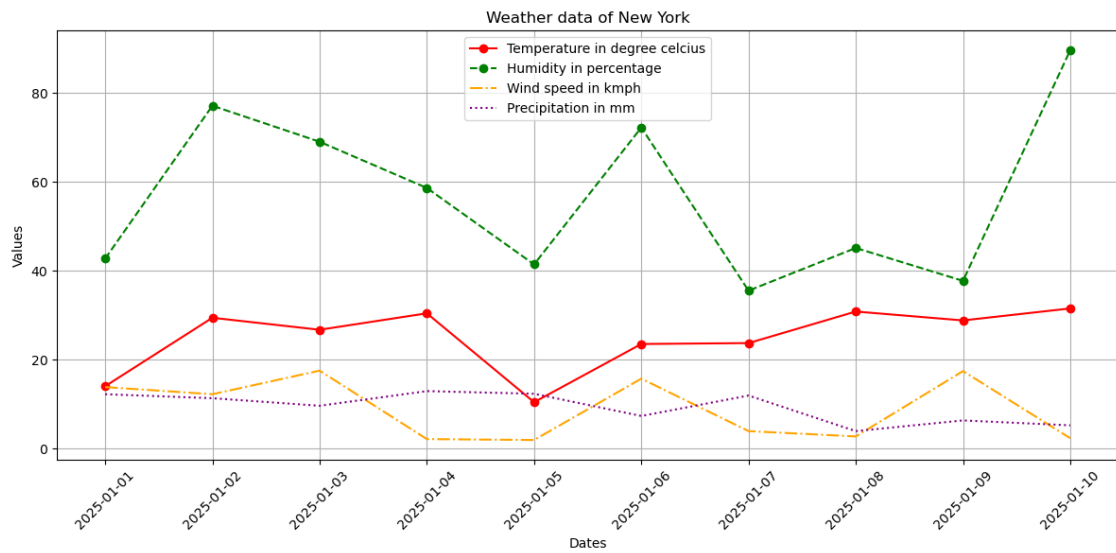
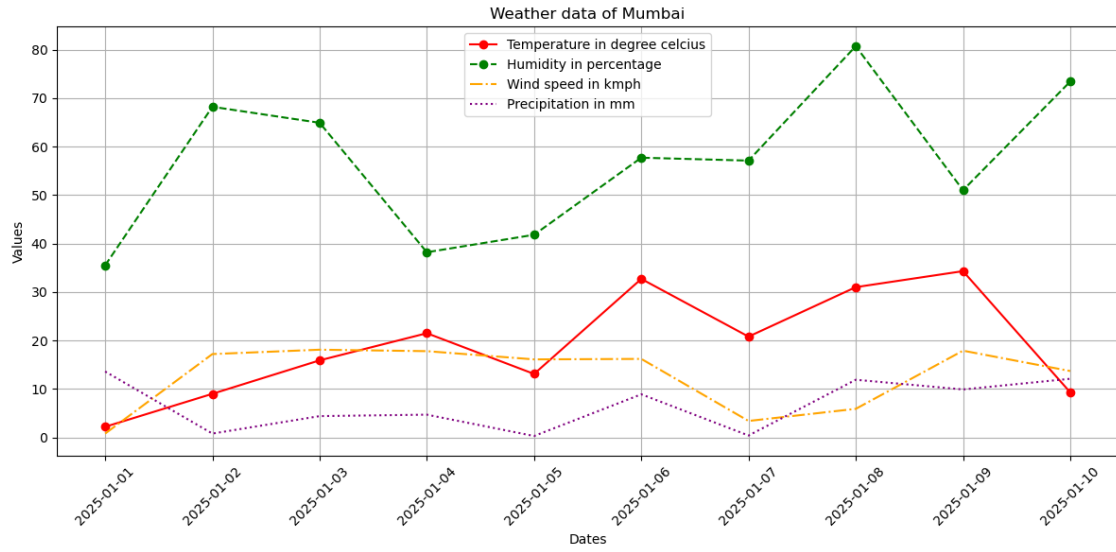
	mean	min	25%	50%	75%	max	std
City							
London	7.14	2.1	3.700	6.95	10.300	14.4	4.180165
Mumbai	6.70	0.3	1.700	6.80	11.400	13.6	5.202991
New York	9.29	3.9	6.550	10.45	12.125	12.9	3.336482
Sydney	8.07	3.9	5.575	7.05	11.525	12.6	3.309935
Tokyo	8.45	2.6	5.975	9.00	10.100	14.9	3.608709

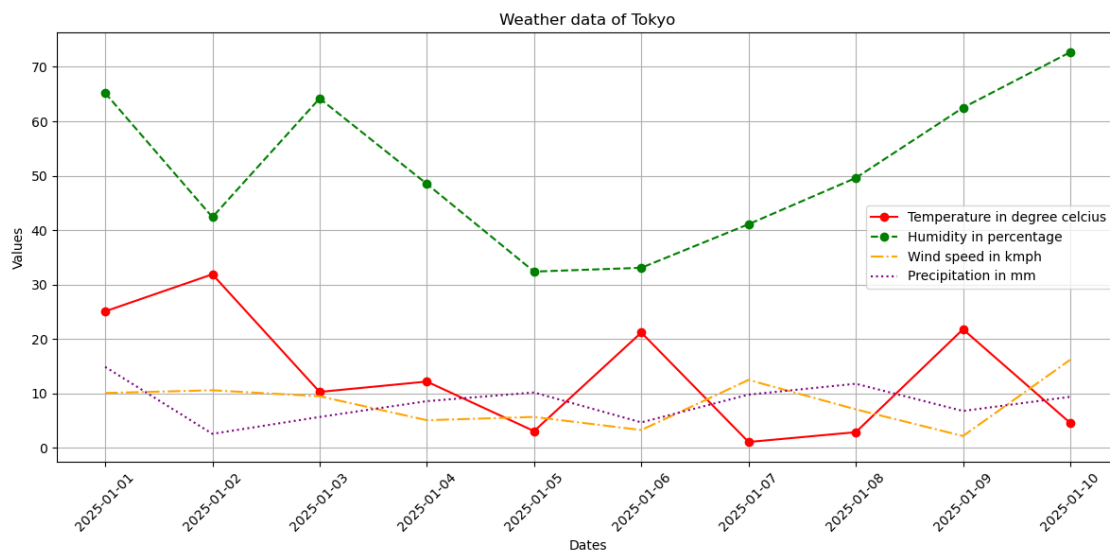
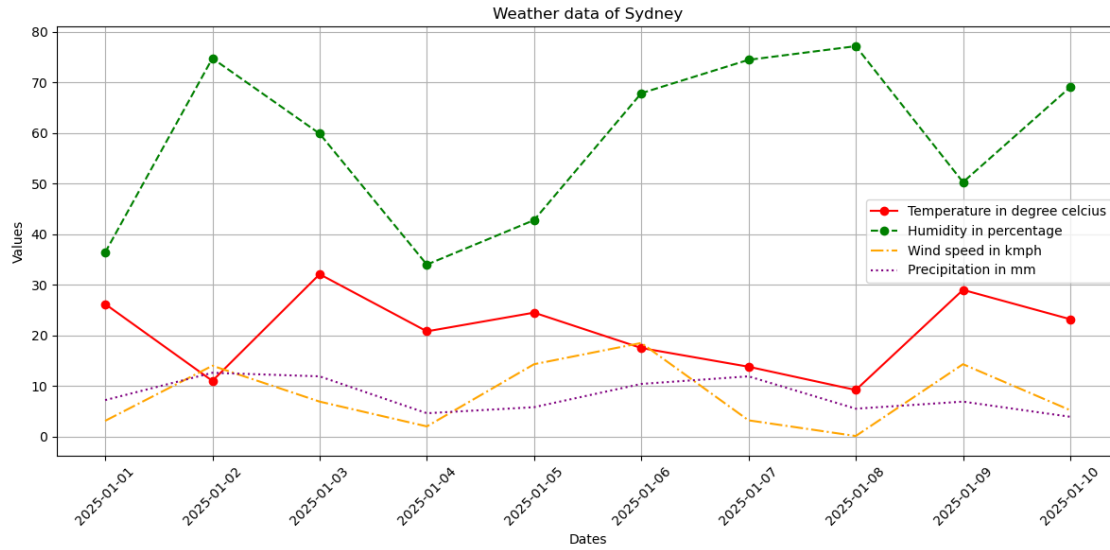
[5 rows x 40 columns]

1.9 If you want to plot the temperature , windspeed precipitation and humidity in each of the cities in the group_by object then you can use the plot function from matplotlib

```
[43]: from matplotlib import pyplot as plt
for city, city_df in group_by_df2:
    plt.figure(figsize=(12,6))
    plt.
    ↪plot(city_df["Date"],city_df["Temperature_C"],color="red",linestyle="--",marker="o",label="T
    ↪in degree celcius")
    plt.
    ↪plot(city_df["Date"],city_df["Humidity_%"],color="green",linestyle="--",marker="o",label="H
    ↪in percentage")
    plt.
    ↪plot(city_df["Date"],city_df["WindSpeed_kmph"],color="orange",linestyle="-.
    ↪",label="Wind speed in kmph")
    plt.
    ↪plot(city_df["Date"],city_df["Precipitation_mm"],color="purple",linestyle=":
    ↪",label="Precipitation in mm")
    plt.title(f"Weather data of {city}")
    plt.xlabel("Dates")
    plt.ylabel("Values")
    plt.xticks(rotation=45)
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()
```







1.9.1 Plot all the temperature from all the cities in one graph.

Method 1: Create sub dataFrames for each city present in the group_by object of Pandas.

```
[44]: mumbai_df = group_by_df2.get_group("Mumbai")
      mumbai_df
```

```
[44]:
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
30	Mumbai	2025-01-01	2.2	35.5	0.8	
31	Mumbai	2025-01-02	9.0	68.2	17.2	

32	Mumbai	2025-01-03	15.9	64.9	18.1
33	Mumbai	2025-01-04	21.5	38.2	17.8
34	Mumbai	2025-01-05	13.1	41.8	16.1
35	Mumbai	2025-01-06	32.7	57.7	16.2
36	Mumbai	2025-01-07	20.8	57.1	3.4
37	Mumbai	2025-01-08	31.0	80.7	5.9
38	Mumbai	2025-01-09	34.3	51.0	17.9
39	Mumbai	2025-01-10	9.3	73.4	13.7

	Precipitation_mm
30	13.6
31	0.8
32	4.4
33	4.7
34	0.3
35	8.9
36	0.4
37	11.9
38	9.9
39	12.1

```
[45]: tokyo_df = group_by_df2.get_group("Tokyo")
      tokyo_df
```

	City	Date	Temperature_C	Humidity_%	WindSpeed_kmph	\
20	Tokyo	2025-01-01	25.1	65.2	10.1	
21	Tokyo	2025-01-02	31.9	42.4	10.6	
22	Tokyo	2025-01-03	10.3	64.2	9.5	
23	Tokyo	2025-01-04	12.2	48.5	5.1	
24	Tokyo	2025-01-05	3.1	32.4	5.7	
25	Tokyo	2025-01-06	21.2	33.1	3.3	
26	Tokyo	2025-01-07	1.1	41.1	12.5	
27	Tokyo	2025-01-08	2.9	49.6	7.1	
28	Tokyo	2025-01-09	21.8	62.5	2.2	
29	Tokyo	2025-01-10	4.6	72.7	16.2	

	Precipitation_mm
20	14.9
21	2.6
22	5.7
23	8.6
24	10.2
25	4.7
26	9.8
27	11.8
28	6.8
29	9.4

```
[46]: sydney_df = group_by_df2.get_group("Sydney")
sydney_df
```

```
[46]:      City      Date  Temperature_C  Humidity_%  WindSpeed_kmph  \
40  Sydney 2025-01-01          26.2         36.4           3.1
41  Sydney 2025-01-02          11.0         74.8          14.0
42  Sydney 2025-01-03          32.1         59.9           6.9
43  Sydney 2025-01-04          20.8         34.0           2.0
44  Sydney 2025-01-05          24.5         42.8          14.3
45  Sydney 2025-01-06          17.5         67.9          18.5
46  Sydney 2025-01-07          13.8         74.5           3.2
47  Sydney 2025-01-08           9.2         77.2           0.1
48  Sydney 2025-01-09          29.0         50.3          14.3
49  Sydney 2025-01-10          23.2         69.1           5.2

      Precipitation_mm
40              7.2
41             12.6
42             11.9
43              4.6
44              5.8
45             10.4
46             11.9
47              5.5
48              6.9
49              3.9
```

```
[47]: london_df = group_by_df2.get_group("London")
london_df
```

```
[47]:      City      Date  Temperature_C  Humidity_%  WindSpeed_kmph  \
10  London 2025-01-01          13.3         42.9          11.4
11  London 2025-01-02           0.5         43.1           7.8
12  London 2025-01-03           7.9         84.0           0.6
13  London 2025-01-04          11.9         53.0           7.3
14  London 2025-01-05           0.9         65.2           1.6
15  London 2025-01-06           0.8         65.7           2.4
16  London 2025-01-07          13.7         60.8          14.8
17  London 2025-01-08          10.1         80.1          17.2
18  London 2025-01-09          30.8         58.4          15.1
19  London 2025-01-10           9.2         46.8           0.3

      Precipitation_mm
10             14.4
11              2.1
12              3.7
13              3.7
```


14	10.7
15	11.2
16	6.0
17	2.6
18	7.9
19	9.1

```
[48]: new_york_df = group_by_df2.get_group("New York")
new_york_df
```

```
[48]:      City      Date  Temperature_C  Humidity_%  WindSpeed_kmph  \
0  New York 2025-01-01           14.0         42.7           13.8
1  New York 2025-01-02           29.4         77.1           12.2
2  New York 2025-01-03           26.7         69.0           17.5
3  New York 2025-01-04           30.4         58.6            2.1
4  New York 2025-01-05           10.4         41.4            1.9
5  New York 2025-01-06           23.5         72.2           15.7
6  New York 2025-01-07           23.7         35.5            3.9
7  New York 2025-01-08           30.8         45.1            2.7
8  New York 2025-01-09           28.8         37.7           17.4
9  New York 2025-01-10           31.5         89.6            2.3
```

	Precipitation_mm
0	12.2
1	11.3
2	9.6
3	12.9
4	12.3
5	7.3
6	11.9
7	3.9
8	6.3
9	5.2

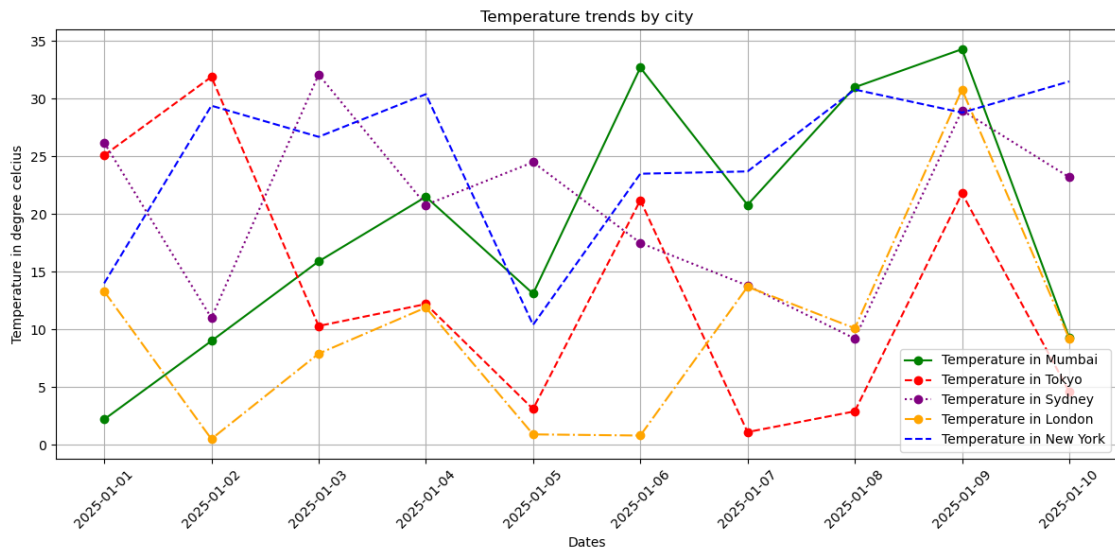
Plotting temperature of each cities

```
[49]: plt.figure(figsize=(12,6))
plt.
    ↳plot(mumbai_df["Date"],mumbai_df["Temperature_C"],color="green",marker="o",linestyle="--",label=
    ↳in Mumbai")
plt.
    ↳plot(tokyo_df["Date"],tokyo_df["Temperature_C"],color="red",marker="o",linestyle="--",label=
    ↳in Tokyo")
plt.
    ↳plot(sydney_df["Date"],sydney_df["Temperature_C"],color="purple",marker="o",linestyle=":
    ↳",label="Temperature in Sydney")
```

```

plt.
    ↪plot(london_df["Date"],london_df["Temperature_C"],color="orange",marker="o",linestyle="-.",
    ↪,label="Temperature in London")
plt.
    ↪plot(new_york_df["Date"],new_york_df["Temperature_C"],color="blue",linestyle="--",label="Te
    ↪in New York")
plt.title("Temperature trends by city")
plt.xlabel("Dates")
plt.ylabel("Temperature in degree celcius")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.legend()
plt.show()

```



Method 2: Use for loop instead of creating separate dataFrames using get_group function

```

[50]: from matplotlib import pyplot as plt
import itertools
# line colors array
colors = ["red", "blue", "green", "orange", "purple", "brown", "cyan", "
    ↪magenta"]
# line variety array
linestyles = ["-", "--", ":", "-.", (0,(3,5,1,5)), (0,(5,10))]

# create infinite cycling iterators
color_cycle = itertools.cycle(colors)
linestyle_cycle = itertools.cycle(linestyles)

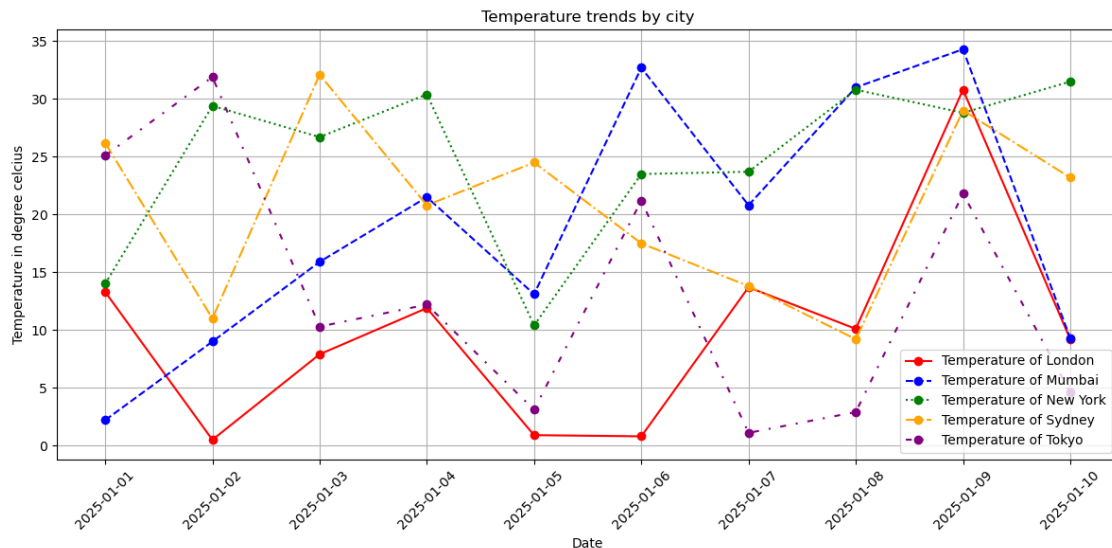
```

```

plt.figure(figsize=(12,6))

for city, city_df in group_by_df2:
    plt.plot(city_df["Date"], city_df["Temperature_C"],marker="o",
             color=next(color_cycle), linestyle=next(linestyle_cycle), label =_
             ↪f"Temperature of {city}")
    )
plt.title("Temperature trends by city")
plt.xlabel("Date")
plt.ylabel("Temperature in degree celcius")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.legend()
plt.show()

```



1.9.2 Explanation :

What are itertools? itertools is a standard python library module that provides fast memory-efficient tools for creating and using iterators. It's often used when: - You want to loop over items infinitely (cycle) - You need combinations or permutations (combinations, permutations) - You want to filter or group data efficiently It's greate for : - Repeating patterns (color, styles) - Complex loops without creating big lists - Lazy evaluations (only generating what's needed) No installation required when it comes this itertools module.

What is itertools.cycle() `itertools.cycle(iterable)` creates an infinite it-erator that loops over the given interable forever. syntax : `cycle_value = itertools.cycle(a_list_of_value)` This takes in a list as a parameter and it will keep

giving you the values from the list one by one then when it reaches the last element in the list it then again goes back to the first element in the list and this cycle keeps on repeating itself infinitely. Here `cycle_value` is essentially an `iterator`

What is `next()` ? `next(iterator)` is a built-in python function that: - Gets the next item from an iterator. - Moves the iterator's internal position forward. If you call `next()` after it's exhausted (and It's not a cycle), you will get a `StopIteration` error.

when to use `itertools.cycle()`? Use `cycle()` when: - You want to reuse a set of values indefinitely without manually resetting an index. - Examples: - Rotating plot colors/styles - cycles through player turns in a game - repeating an animation sequence - periodic events in simulations

Method 2 is much more efficient when compared to method 1 since I don't have to manually get all the dataFrames by city and plot the graph.

1.9.3 Plot all the WindSpeed from all the cities in one graph.

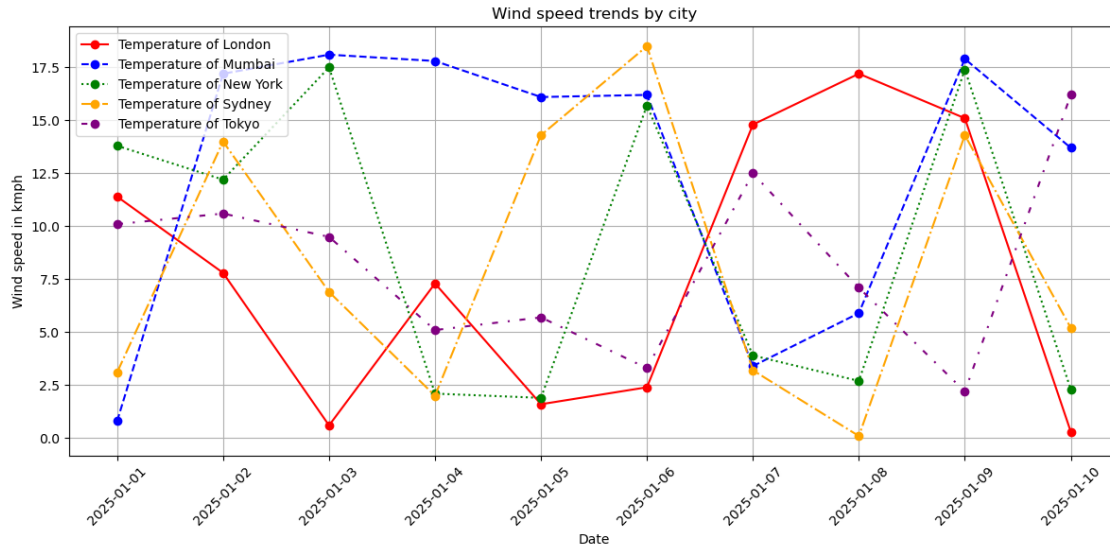
Here I will be using method 2 since the it's more efficient.

```
[51]: # line colors array
color = ["red","blue","green","orange","purple","brown","cyan","magenta"]
# line variety array
linestyles = ["-", "--", ":", "-.", (0,(3,5,1,5)), (0,(5,10))]

# create an infinite cycling iterators
color_cycle = itertools.cycle(color)
linestyle_cycle = itertools.cycle(linestyles)

plt.figure(figsize = (12,6))

for city,city_df in group_by_df2:
    plt.
        plot(city_df["Date"],city_df["WindSpeed_kmph"],marker="o",color=next(color_cycle),linestyle
            label=f"Temperature of {city}")
plt.title("Wind speed trends by city")
plt.xlabel("Date")
plt.ylabel("Wind speed in kmph")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.legend()
plt.show()
```



1.9.4 Plot all the Humidity in percentage from all the cities in one graph.

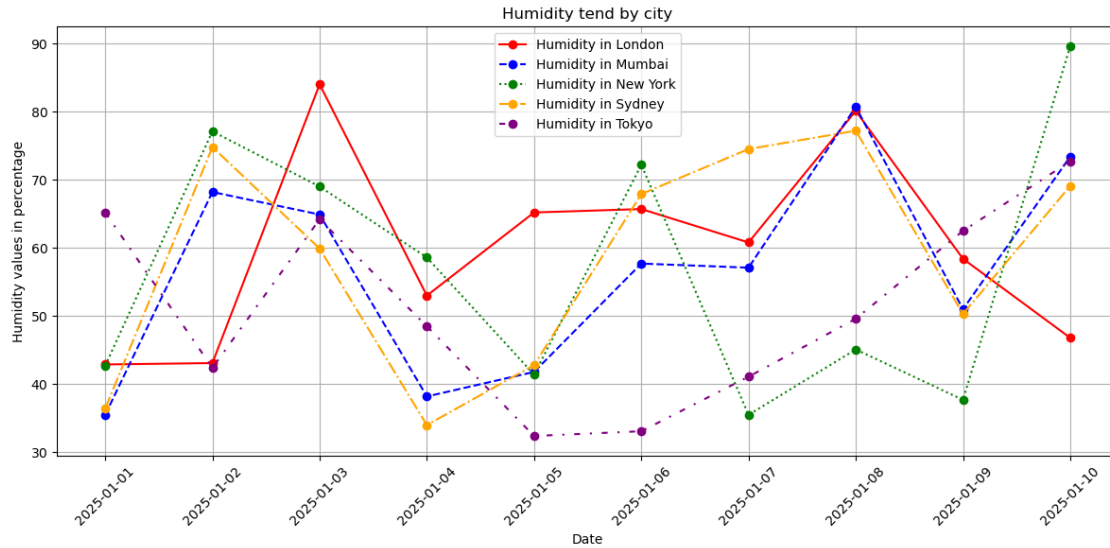
```
[67]: # list of colors
color = ["red", "blue", "green", "orange", "purple", "brown", "cyan", "magenta"]
# list of linestyle
linestyle = ["-", "--", ":", "-.", (0, (3, 5, 1, 5)), (0, (5, 10))]

# create infinite cycling iterators
color_cycle = itertools.cycle(color)
linestyle_cycle = itertools.cycle(linestyle)

plt.figure(figsize=(12, 6))

for city, city_df in group_by_df2:
    plt.
        plot(city_df["Date"], city_df["Humidity_%"], marker="o", color=next(color_cycle),
            linestyle = next(linestyle_cycle), label=f"Humidity in {city}")

plt.title("Humidity tend by city")
plt.xlabel("Date")
plt.ylabel("Humidity values in percentage")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.legend()
plt.show()
```



1.9.5 Plot Precipitation from all cities in one graph

```
[53]: # list of colors
colors = ["red", "blue", "green", "orange", "purple", "brown", "cyan", "mergenta"]

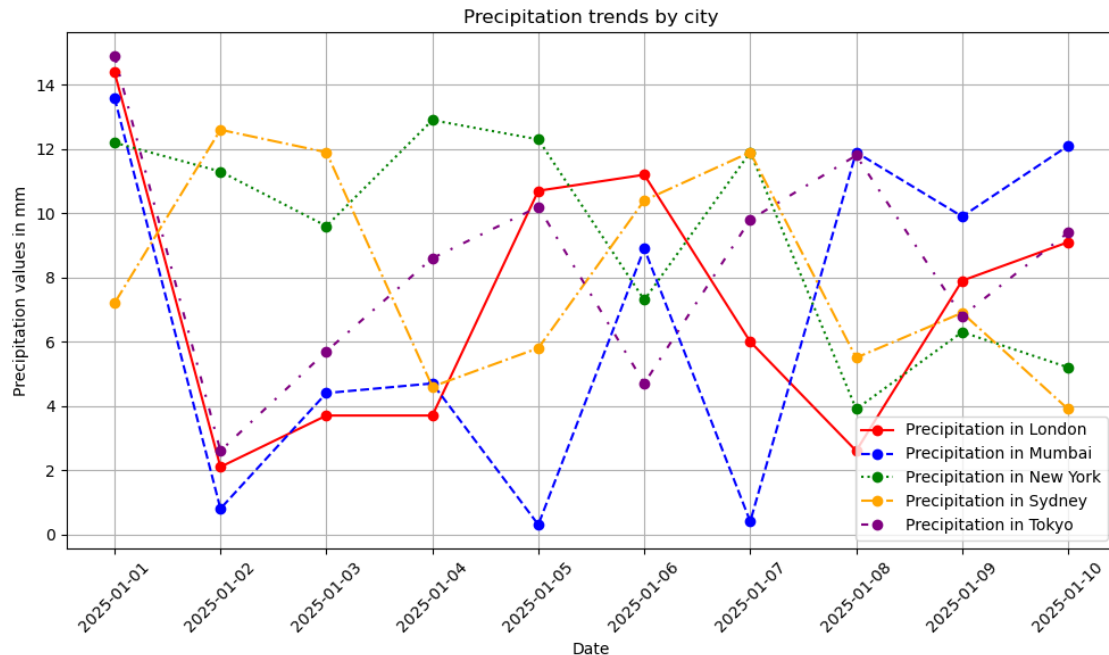
# list of linestyle
linestyles = ["-", "--", ":", "-.", (0,(3,5,1,5)),(0,(5,10))]

# create an infinite cycling iterators
color_cycle = itertools.cycle(colors)
linestyle_cycle = itertools.cycle(linestyles)

plt.figure(figsize=(10,6))

for city, city_df in group_by_df2:
    plt.plot(city_df["Date"],city_df["Precipitation_mm"],marker = "o", color =next(color_cycle), linestyle = next(linestyle_cycle),
            label = f"Precipitation in {city}")

plt.title("Precipitation trends by city")
plt.xlabel("Date")
plt.ylabel("Precipitation values in mm")
plt.xticks(rotation=45)
plt.tight_layout()
plt.grid(True)
plt.legend()
plt.show()
```



1.9.6 As you can see in the above examples where data for example temperature , windspeed , humidity and precipitation are being plotted for all the cities in a single graph. But the graph looks a little bit messy to be able to make something out

1.9.7 Solution to this problem:

Alternatively I can try to use these methods : - Subplots per city - Smoothing the lines (Rolling average) - Bar Chart

1.9.8 Sub plots per city

Plot the subplots for temperature for all the cities

```
[54]: # create a cities list using the city keys in the group_by object of pandas
cities =list(group_by_df2.groups.keys())
# create figure and axes
fig, axes = plt.subplots(len(cities), 1, figsize = (10, 2 * len(cities)),
    ↳sharex = True)

color_list = ["red","blue","purple","green","magenta","orange"]
linestyle_list = ["-", "--", ":", "-.", (0,(3,5,1,5)), (0,(5,10))]

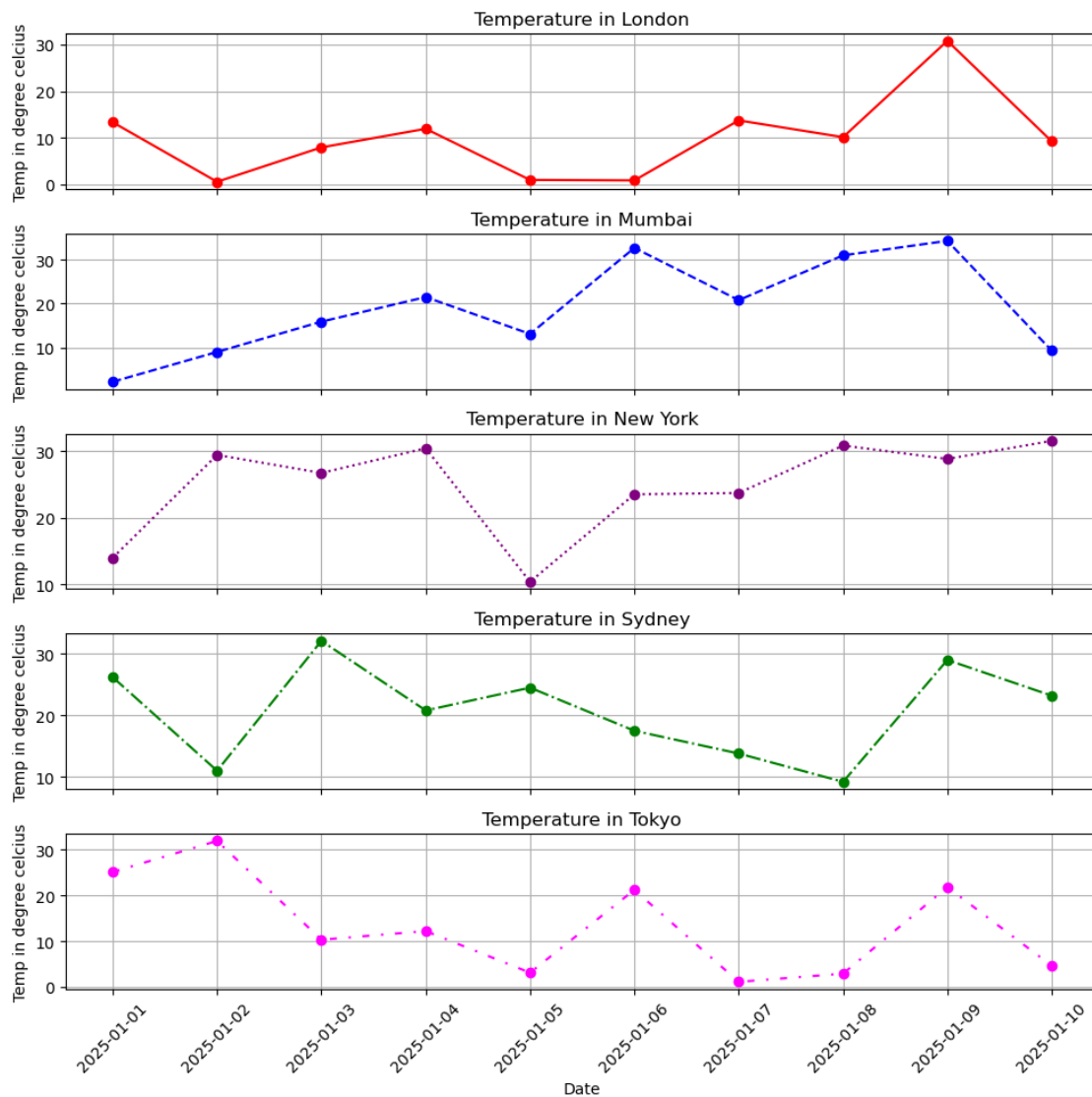
# create a color_cycle iterator
color_cycle = itertools.cycle(color_list)
linestyle_cycle = itertools.cycle(linestyle_list)
```

```

for ax, (city, city_df) in zip(axes, group_by_df2):
    ax.plot(city_df["Date"], city_df["Temperature_C"], marker = "o",
            linestyle=next(linestyle_cycle), color=next(color_cycle))
    ax.set_title(f"Temperature in {city}")
    ax.set_ylabel("Temp in degree celcius")
    ax.grid(True)

plt.xlabel("Date")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



1.9.9 Explanation:

```
list(group_by_df2.groups.keys())
```

- `group_by_df2` is a `DataFrameGroupBy` objects created by:
- `.groups` contains a dictionary where :
 - Keys = unique group labels (city names in this case)
 - Values = index positions of rows in each group
- ```
group_by_df2.groups
```

```
{
```

```
 'Delhi': Int64Index([0, 5, 9], dtype='int64'),
```

```
 'Mumbai': Int64Index([1, 4, 7], dtype='int64'),
```

```
 ...
```

```
}
```

  - `.group.keys()` returns the dictionary keys (`dict_keys` object) -> `["Delhi", "Mumbai" ...]`.
  - Wrapping with `list()` turns that into a Python list.

```
fig, axes = plt.subplots(len(cities), 1, figsize=(10, 2*len(cities)),
sharex=True)
```

- `plt.subplots(nrows, ncols, ...)` creates multiple subplots in a single figure.
- `len(cities)` -> number of rows in the grid (one row per city)
- `1` -> only 1 column
- `figsize=(10, 2*len(cities))`
  - width = 10 inches.
  - height = 2 x number of cities -> more rows -> taller figure.
- `sharex=True` means all subplots share the same X-axis, so zooming/panning affects all of them.
- python

```
fig, axes = plt.subplots(nrows=1,
ncols=1, *, sharex=False, sharey=False,
squeeze=True, subplot_kw=None, gridspec_kw=None,
**fig_kw)
```

  - `nrows` :
    - \* Type : int
    - \* Meaning : Number of rows of subplots
    - \* Default : 1
    - \* Example : `plt.subplot(nrows=2, ncols=1)`
  - `ncols` :
    - \* Type : int
    - \* Meaning : Number of columns of subplots
    - \* Default : 1
    - \* Example : `plt.subplots(nrows=1, ncols=1)`
  - `sharex` :
    - \* Type : bool or {'none', 'all', 'row', 'col'}
    - \* Meaning : Weather subplots share the same x-axis.
    - \* Default : False

- \* Options :
  - True / 'all' : All plots use the same x-axis ticks/limits.
  - 'row' : Plots in the same row share the x-axis
  - 'col' : Plots in the same column share the x-axis
  - False / 'none': No sharing
- \* Example : `plt.subplots(2,2,sharex='col')`
- sharey :
  - \* Type : bool or {'none', 'all', 'row', 'col'}
  - \* Meaning : Whether subplots share the same y-axis.
  - \* Works the same way as sharex
- squeeze :
  - \* Type : bool
  - \* Meaning : If True (default), it removes extra dimensions from the axes array when possible.
  - \* Example : `fig, axes = plt.subplots(1,1)`
    - With squeeze = True : The 'axes' is a single axes object
    - With squeeze = False : 'axes' is always a 2D array (`axes[0,0]`)
- subplot\_kw :
  - \* Type : dict
  - \* Meaning : Keyword argument passes to each subplot's `add_subplot()` call
  - \* Example : `plt.subplots(2,2,subplot_kw={'projection' : 'polar'})` → This will create 4 polar coordinate plots
- gridspec\_kw :
  - \* Type : dict
  - \* Meaning : Extra keyword args for the underlying `GridSpec` (spacing of subplots)
  - \* Example : `plt.subplots(2,2,gridspec_kw = {'width_ratios' : [2,1]})`
- \*\*fig\_kw :
  - \* Meaning : Extra keyword args for `plt.figure()` (size,DPI,etc.)
  - \* Example : `plt.subplots(2,2,figsize = (10,6),dpi=120)`

```
for ax, (city, city_df) in zip(axes, group_by_df2):
```

- `group_by_df2` is iterable → each iteration gives (`group_name`, `group_df`).
  - `group_name` → example "Delhi".
  - `group_df` → DataFrame containing only that city's rows.
  - `axes` is a Numpy array of subplot objects (one for each city)
  - `zip(axes,group_by_df2)` pairs:
    - \* `(axes[0], ("Delhi",Delhi_df))`
    - \* `(axes[1], ("Mumbai",Mumbai_df))`
- draws each city's precipitation in its own subplot.

**What is ax?**

- `ax` is an Axes object from Matplotlib.
- It's the object that actually represents a single subplot in the figure.
- All plotting commands (`plot`, `scatter`, `set_title`, `grid`) are methods of an axes object.
- This means :
  - `ax.plot()` draws on that specific subplot.

- `ax.set_title()` changes the title for that specific subplot. Without `ax`, you would be using `plt.plot()` which always draws on the current subplot not great when working on multiple sub plots.

`plt.tight_layout()`

- Adjust spacing between subplots automatically so :
  - Titles and labels don't overlap.
  - The layout fits nicely into the figure area.

**Plot the subplot for windspeed for all the cities.**

```
[55]: # create a list of cities using the group_by object
cities = list(group_by_df2.groups.keys())

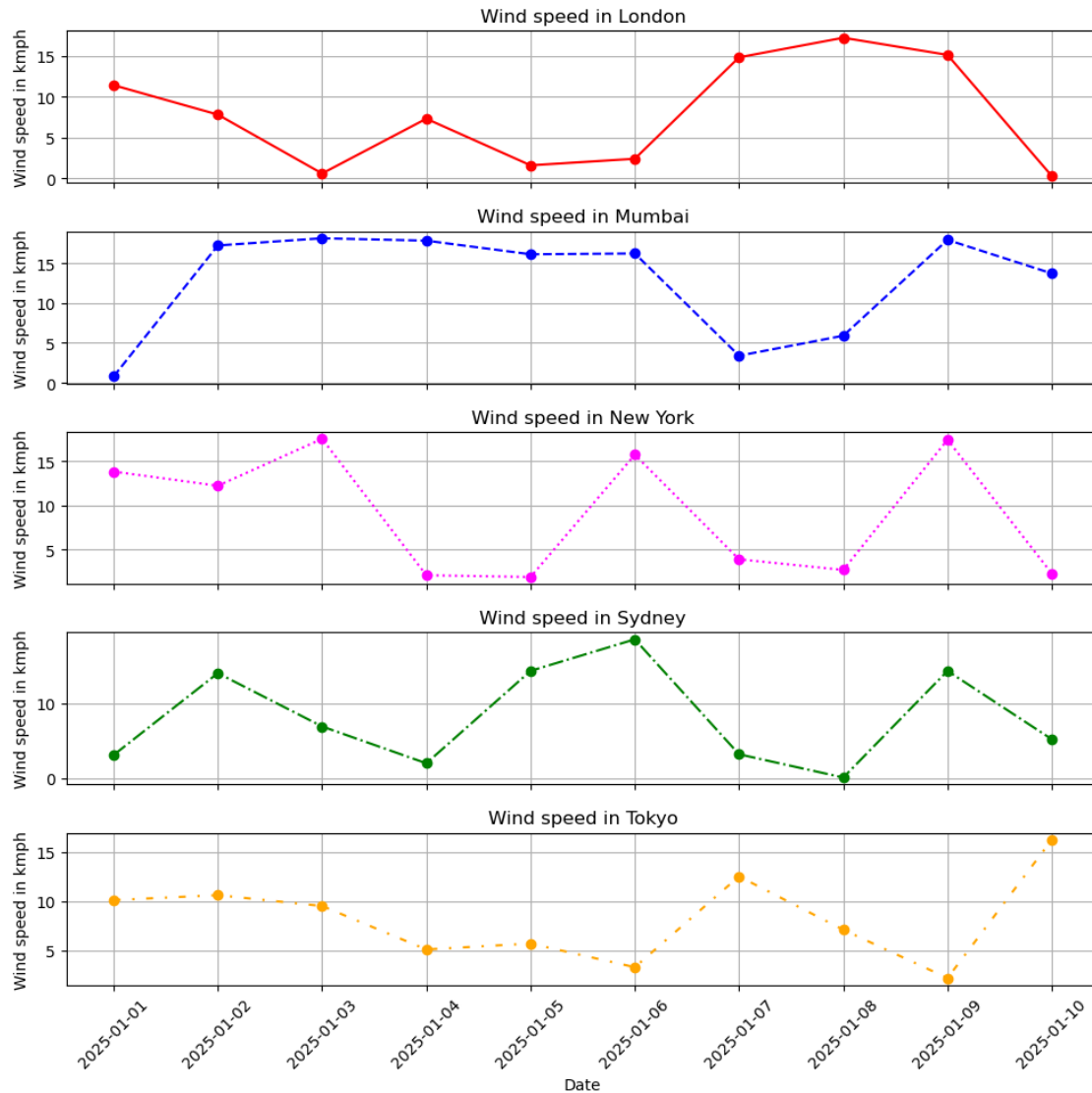
create fig and axes
fig , axes = plt.subplots(len(cities),1,figsize = (10,2 * len(cities)),
 ↳sharex=True)

list of colors
color_list = ["red","blue","magenta","green", "orange", "purple"]
list of linestyles
linestyle = ["-", "--", ":", "-.", (0,(3,5,1,5)), (0,(5,10))]

create a color_cycle iterator
color_cycle = itertools.cycle(color_list)
create a linestyle_cycle iterator
linestyle_cycle = itertools.cycle(linestyle)

for ax, (city,city_df) in zip(axes,group_by_df2):
 ax.
 ↳plot(city_df["Date"],city_df["WindSpeed_kmph"],marker="o",linestyle=next(linestyle_cycle),c
)
 ax.set_title(f"Wind speed in {city}")
 ax.set_ylabel("Wind speed in kmph")
 ax.grid(True)

plt.xlabel("Date")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Plot the subplot for precipitation for all the cities.

```
[56]: # create a list of cities
cities = list(group_by_df2.groups.keys())
create axes and figs
figs, axes = plt.subplots(len(cities),1,figsize =(
 10,2*len(cities)),sharex=True)

create a color list
color_list = ["red","blue","magenta","green","orange","yellow"]
create a linestyle_list
linestyle_list = ["-","--",":","-.",(0,(3,5,1,5)), (0,(5,10))]
```

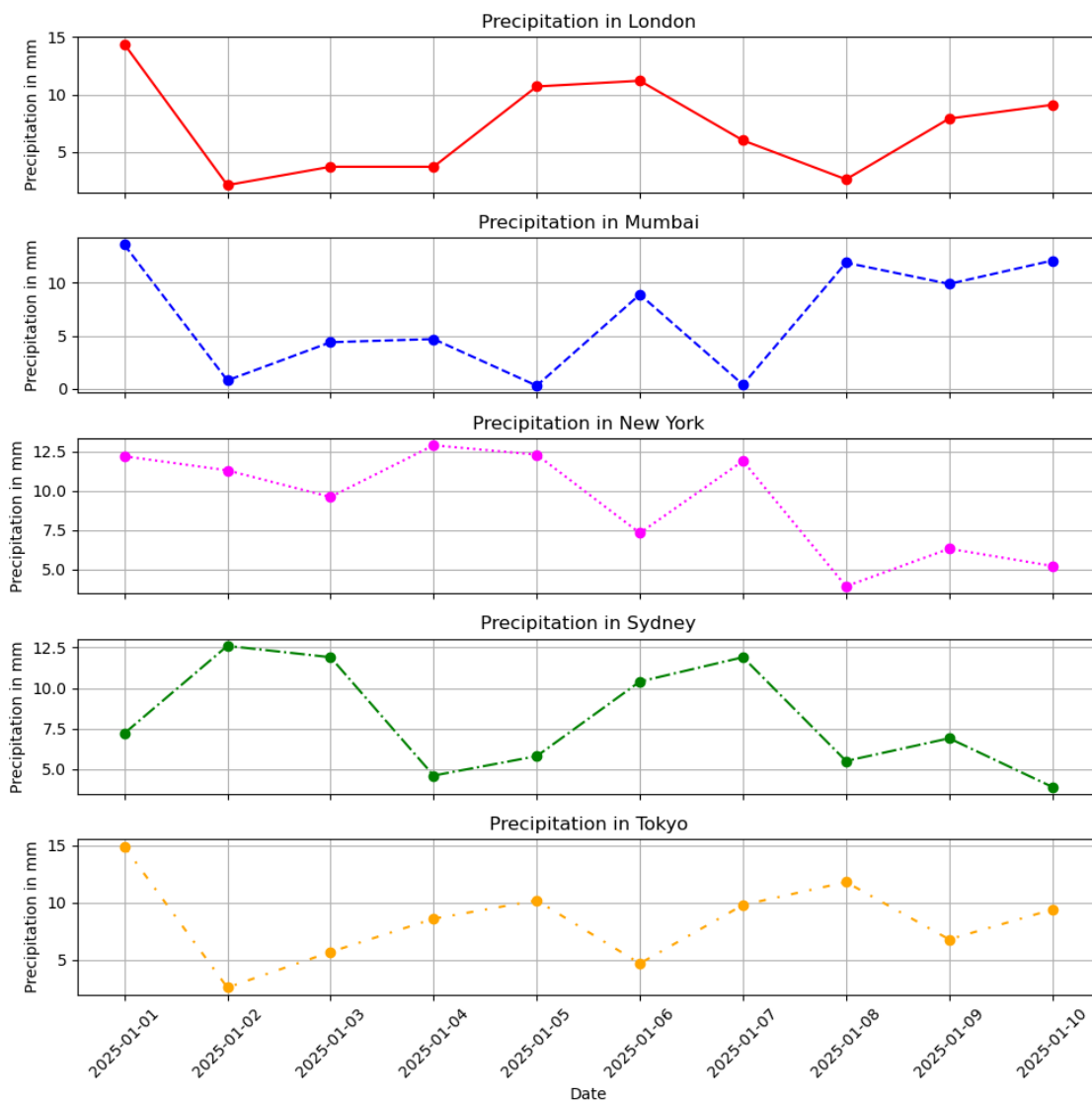
```

create a color cycle using iterator
color_cycle = itertools.cycle(color_list)
linestyle_cycle = itertools.cycle(linestyle_list)

for ax, (city,city_df) in zip(axes, group_by_df2):
 ax.plot(city_df["Date"],city_df["Precipitation_mm"],marker = "o", linestyle_
 ↪= next(linestyle_cycle),color = next(color_cycle))
 ax.set_title(f"Precipitation in {city}")
 ax.set_ylabel("Precipitation in mm")
 ax.grid(True)

plt.xlabel("Date")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```



Plot the subplot for humidity for all the cities.

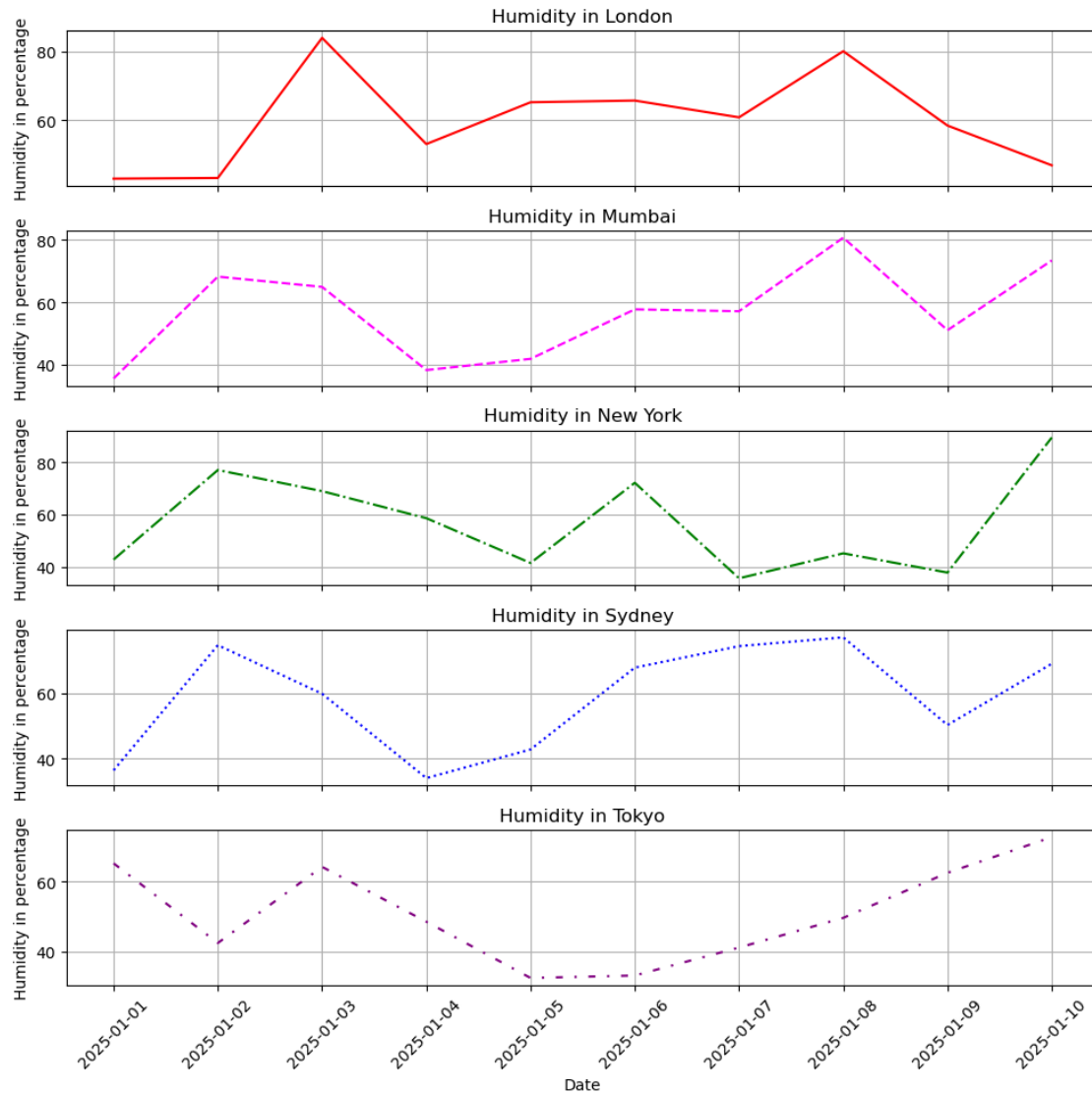
```
[57]: # create a city list
city_list = list(group_by_df2.groups.keys())
create figs and axes
figs , axes = plt.
 ↳subplots(len(city_list),1,figsize=(10,2*len(city_list)),sharex=True)

create color list
color_list = ["red","magenta","green","blue","purple","yellow"]
create a linestyle_list
linestyle_list = ["-", "--", "-.", ":", (0,(3,5,1,5)), (0,(5,10))]

create a color iterator
color_cycle = itertools.cycle(color_list)
create a linestyle iterator
linestyle_cycle = itertools.cycle(linestyle_list)

for ax , (city,city_df) in zip(axes,group_by_df2):
 ax.
 ↳plot(city_df["Date"],city_df["Humidity_%"],linestyle=next(linestyle_cycle),color=next(color
 ax.set_title(f"Humidity in {city}")
 ax.set_ylabel("Humidity in percentage")
 ax.grid(True)

plt.xlabel("Date")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



#### 1.9.10 Pros and cons of sub plot:

- Pros : Clear trends for each city
- Cons : HArder to compare cities directly

#### 1.9.11 Smoothing the lines (Rolling average)

Plot temperature of all the cities (Rolling average) method

```
[58]: # create a list of colors
color_list = ["red", "blue", "magenta", "orange", "green", "purple"]
linestyle_list = ["-", "--", ":", "-.", (0, (3, 5, 1, 5)), (0, (5, 10))]

create a cyclic looping list using iterators
```

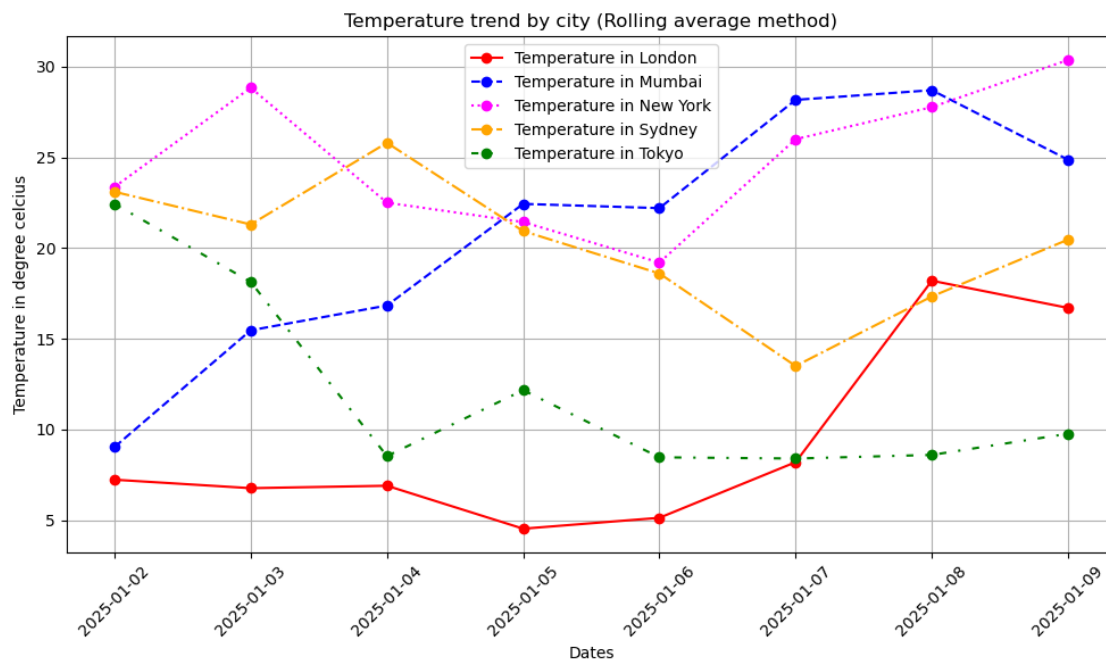
```

color_cycle = itertools.cycle(color_list)
linestyle_cycle = itertools.cycle(linestyle_list)

plt.figure(figsize=(10,6))
for city, city_df in group_by_df2:
 plt.plot(city_df["Date"] , city_df["Temperature_C"].
 rolling(window=3,center=True).mean(),
 marker = "o", color=next(color_cycle),
 linestyle=next(linestyle_cycle),
 label=f"Temperature in {city}")

plt.xlabel("Dates")
plt.ylabel("Temperature in degree celcius")
plt.xticks(rotation=45)
plt.title("Temperature trend by city (Rolling average method)")
plt.tight_layout()
plt.grid(True)
plt.legend()
plt.show()

```



### 1.9.12 Explaiantion :

What is “rolling” in pandas?

- A rolling operation is a way to calculate statistics (like mean, sum standard deviation etc.) over a sliding window of data points.



- Imagine you have a time series you don't always want to calculate stats over the entire series but instead over the last N points at each step.

**What is “rolling average”?** A rolling average (also called moving average) is the mean of data points within that moving window. Example: Data: [2,4,6,8,10] with a window size of 3 ->

```
First_mean = (2+4+6) / 3 = 4
Second_mean = (4+6+8) / 3 = 6
Third_mean = (6+8+10) / 3 = 8
```

This smooths the data by reducing short-term fluctuations and highlighting long-term trends.

**What is window=3?**

- This means the moving window contains 3 consecutive data points at a time.
- For precipitation data:
  - Day 1, 2, 3 -> average
  - Day 2, 3, 4 -> average
  - Day 3, 4, 5 -> average and so on

**What is center=True?**

- By default (center=False), the result for a given row is aligned to the right edge of the window.
- If center = True, the window is centered on the current row. Example with days [1,2,3,4,5]

center=False, window=3:

```
Day 3 → avg of days 1-3
Day 4 → avg of days 2-4
Day 5 → avg of days 3-5
```

center=True, window=3:

```
Day 2 → avg of days 1-3
Day 3 → avg of days 2-4
Day 4 → avg of days 3-5
```

This is especially important for plotting because centered windows align the smoothed value with the middle of the window, making trends easier to interpret visually.

### 1.9.13 Pros and cons of Smoothing the lines (Rolling average):

- Pros : Removes the day to day noise, easier to see patterns.
- Cons : You lose exact daily values.

**Plot humidity of all the cities (Rolling average) method**

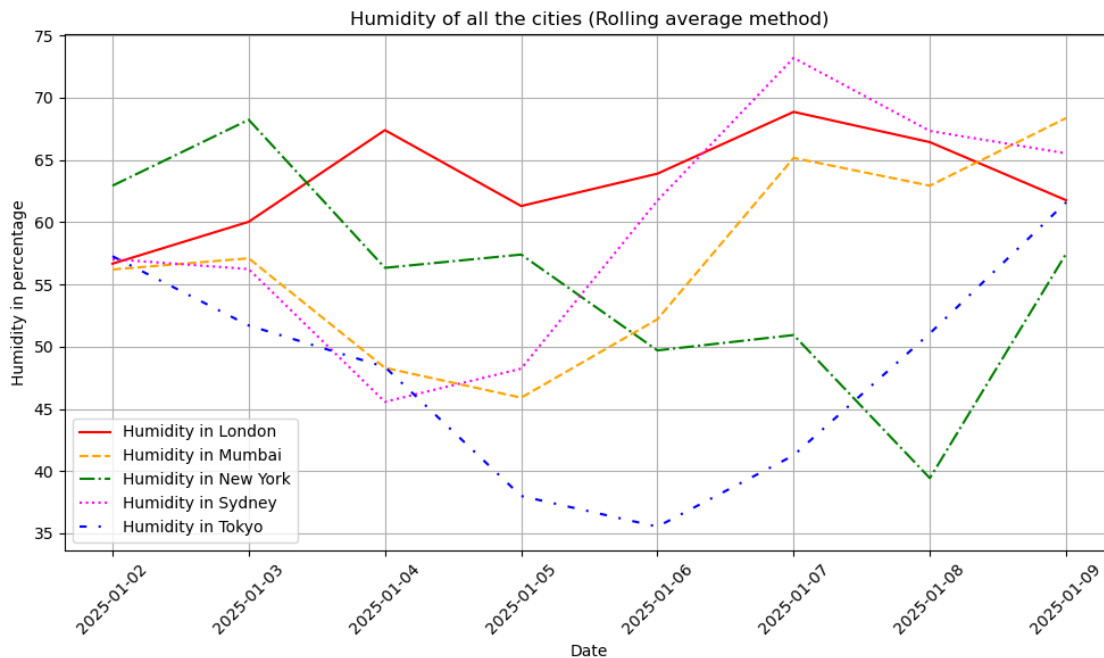
```
[59]: # create the list of colors
color_list = ["red", "orange", "green", "magenta", "blue", "purple"]
create a linestyle list
linestyle_list = ["-", "--", "-.", ":", (0, (3, 5, 1, 5)), (0, (5, 10))]
```

```

create color cycle for iterators
color_cycle = itertools.cycle(color_list)
create linestyle_cycle for iterators
linestyle_cycle = itertools.cycle(linestyle_list)

plt.figure(figsize = (10,6))
for city, city_df in group_by_df2:
 plt.plot(city_df["Date"],city_df["Humidity_%"].
 .rolling(window=3,center=True).mean(),linestyle = next(linestyle_cycle),color=
 next(color_cycle),
 label = f"Humidity in {city}")
plt.title("Humidity of all the cities (Rolling average method)")
plt.xlabel("Date")
plt.ylabel("Humidity in percentage")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.legend()
plt.show()

```



Plot wind speed of all the cities (Rolling average) method

```

[60]: # create a color list
color_list = ["red","orange","green","magenta","blue","purple"]
create a linestyle list

```

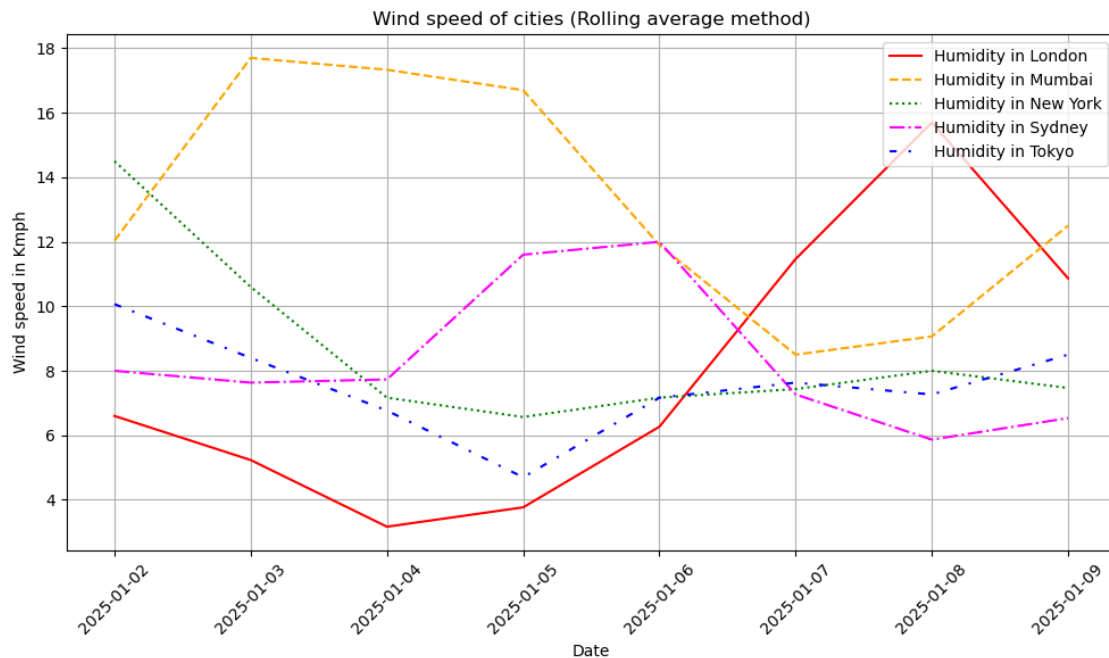
```

linestyle_list = ["-", "--", ":", "-.", (0, (3, 5, 1, 5)), (0, (5, 10))]

create a color cycle for iterators
color_cycle = itertools.cycle(color_list)
create a linestyle cycle for iterators
linestyle_cycle = itertools.cycle(linestyle_list)

plt.figure(figsize = (10,6))
for city, city_df in group_by_df2:
 plt.plot(city_df["Date"],city_df["WindSpeed_kmph"].
 rolling(window=3,center=True).mean(),linestyle = next(linestyle_cycle),
 color = next(color_cycle),label = f"Humidity in {city}")
plt.title("Wind speed of cities (Rolling average method)")
plt.xlabel("Date")
plt.ylabel("Wind speed in Kmph")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.legend()
plt.show()

```



Plot precipitation of all the cities (Rolling average) method.

```

[61]: # create a color list
color_list = ["red", "orange", "magenta", "green", "blue", "purple"]
create a linestyle list

```

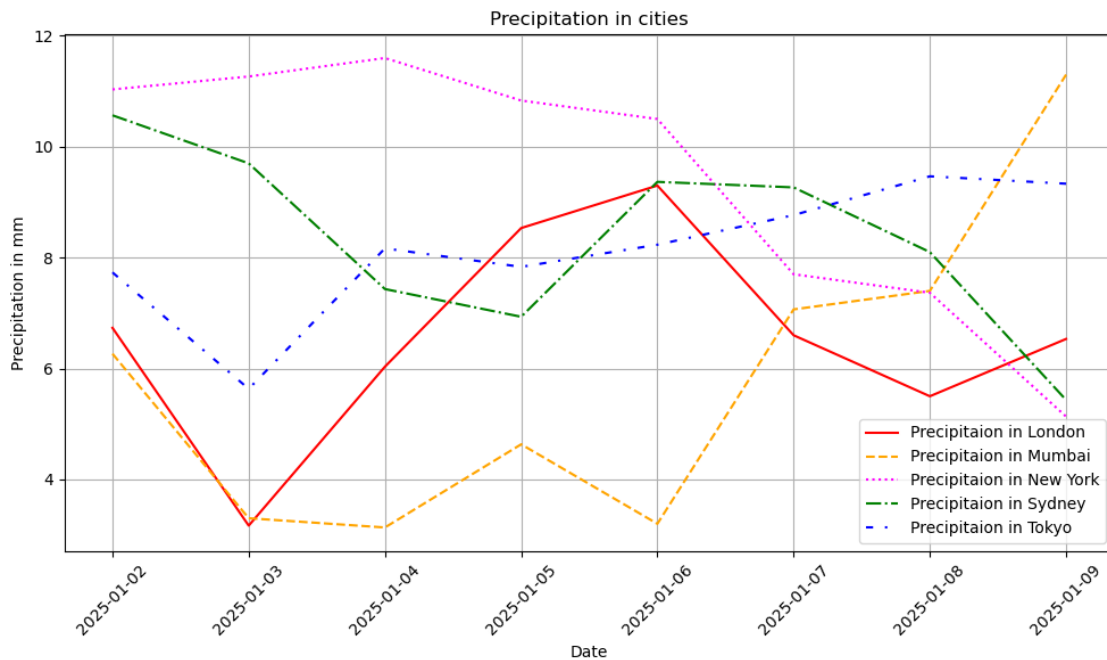
```

linestyle_list = ["-", "--", ":", "-.", (0, (3, 5, 1, 5)), (0, (5, 10))]

create a color_cycle for iterators
color_cycle = itertools.cycle(color_list)
create a linestyle_cycle for iterators
linestyle_cycle = itertools.cycle(linestyle_list)

plt.figure(figsize = (10,6))
for city, city_df in group_by_df2:
 plt.plot(city_df["Date"], city_df["Precipitation_mm"].
 rolling(window=3, center=True).mean(),
 linestyle=next(linestyle_cycle), color=next(color_cycle), label = f"
 f"Precipitaion in {city}")
plt.title("Precipitation in cities")
plt.xlabel("Date")
plt.ylabel("Precipitation in mm")
plt.xticks(rotation = 45)
plt.grid(True)
plt.tight_layout()
plt.legend()
plt.show()

```

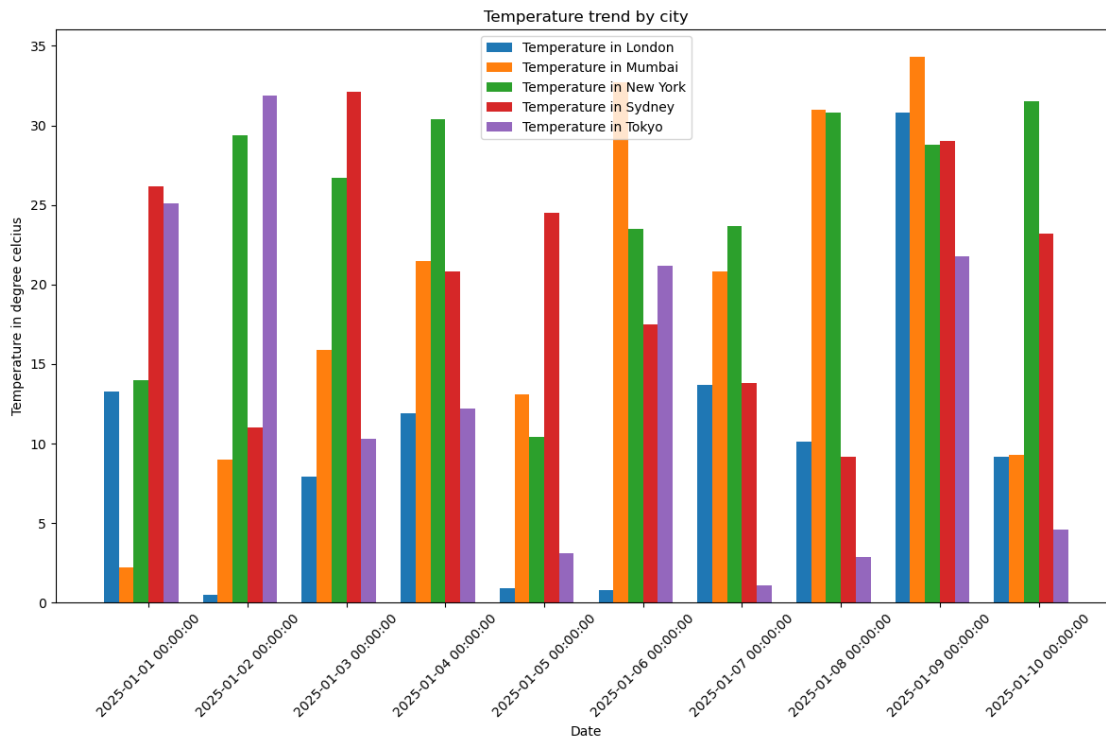


#### 1.9.14 Alternative chart type : Bar chart

Plot the temperature of all the cities using bar chart

```
[62]: import numpy as np
dates = sorted(df2["Date"].unique())
x = np.arange(len(dates))
width = 0.15 # bar width

plt.figure(figsize = (12,8))
for i, (city,city_df) in enumerate(group_by_df2):
 plt.bar(
 x + i*width,
 city_df["Temperature_C"],
 width = width,
 label = f"Temperature in {city}"
)
plt.title("Temperature trend by city")
plt.xlabel("Date")
plt.ylabel("Temperature in degree celcius")
plt.xticks(x + width*(len(group_by_df2)/2),dates,rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```



### 1.9.15 Explanation :

What `df2["Date"].unique()` does?

- gets all the unique dates in the dataset.
- `sorted(...)` -> sorted them in ascending order (so they plot in chronological order).
- `np.arange(len(dates))` -> creates evenly spaced numeric positions for each data example [0,1,2,3, ... ] This is necessary because bar charts need numeric x-coordinates.
- `width = 0.15` -> defines the thickness of each bar. Smaller width means thinner bar. ##### Why? We need a numerical x-axis for plotting. Dates themselves can't be directly offset mathematically, so we use index position(0,1,2 ... ) and later map them back to date labels.

`enumerate(...)` It attaches an index `i` to each iteration: - `i` is 0 for first city, 1 for second city and so on.

```
plt.bar(x + i*width,city_df["Temperature_C"],width=width,label=f"{city}")
```

- `x+i*width` -> It shifts the bars for each city sideways:
  - If all the cities used `x` as the x-position, they'd overlap.
  - Adding `i*width` moves each city's bars slightly to the right for grouping.
- `city_df["Temperature_C"]` -> height of the bars.
- `width=width` -> Thickness of the bars.
- `label = f"{city}"` -> name of the city of the legend.

```
plt.xticks(x + width*(len(group_by_df2)/2), dates, rotation=45)
```

- `x + width*(len(group_by_df2)/2)` -> centers the x-tick labels under the group of bars for each date:
  - Without this, ticks would appear under the first city's bar, not in the middle of the group.
  - `len(group_by_df2)` -> number of cities (groups).
  - Multiplying by `width/2` shifts ticks to the middle of the group.
- `dates` -> replaces numeric positions with actual date strings. ##### Why? Without centering the labels would look visually "misaligned" with the grouped bars.

## Pros and cons of Bar chart

- Pros : Very readable for smaller dataset, easy to compare per date.
- Cons : Gets wide if you have many cities.

## Plot the humidity of all the cities using bar chart

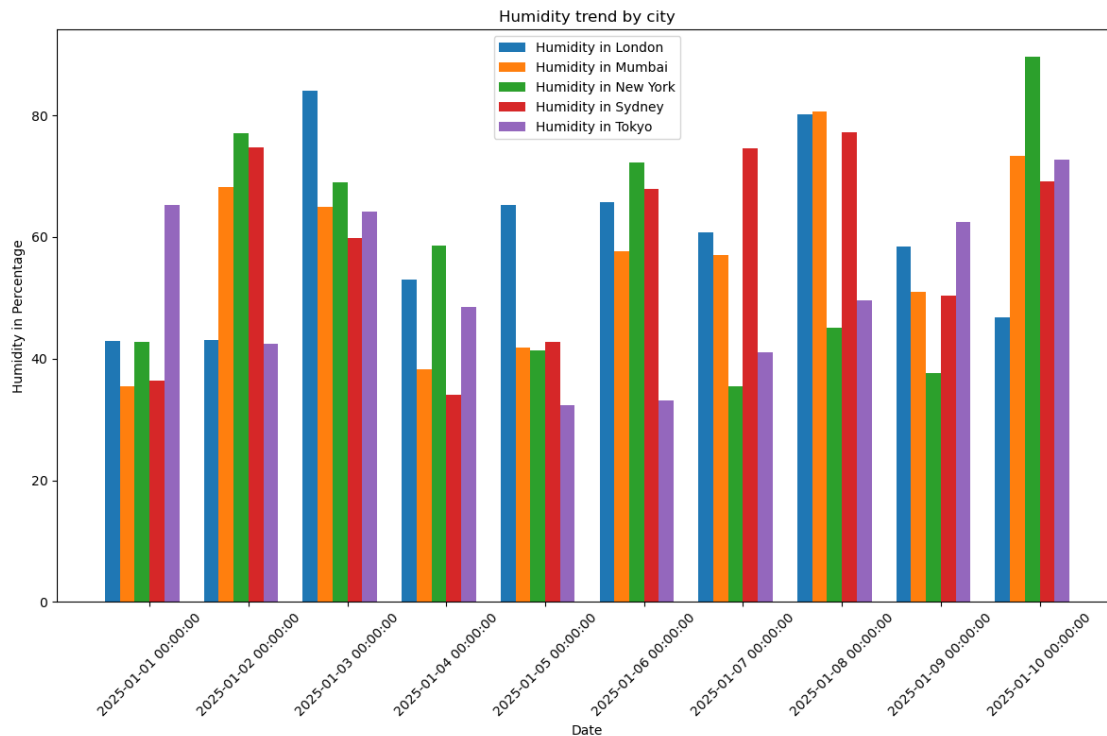
```
[63]: # get all the unique dates and then sort it chronologically
dates = sorted(df2["Date"].unique())
x = np.arange(len(dates))
width = 0.15

plt.figure(figsize=(12,8))
for i, (city,city_df) in enumerate(group_by_df2):
 plt.bar(
 x + i*width,
 city_df["Humidity_%"],
 width = width,
```

```

 label = f"Humidity in {city}"
)
plt.title("Humidity trend by city")
plt.xlabel("Date")
plt.ylabel("Humidity in Percentage")
plt.xticks(x+width*(len(group_by_df2)/2),dates,rotation=45)
plt.tight_layout()
plt.legend()
plt.show()

```



### Plot the WindSpeed of all the cities using bar chart

```

[64]: # get all unique dates and then sort them chronologically
dates = sorted(df2["Date"].unique())
x = np.arange(len(dates))
width = 0.15

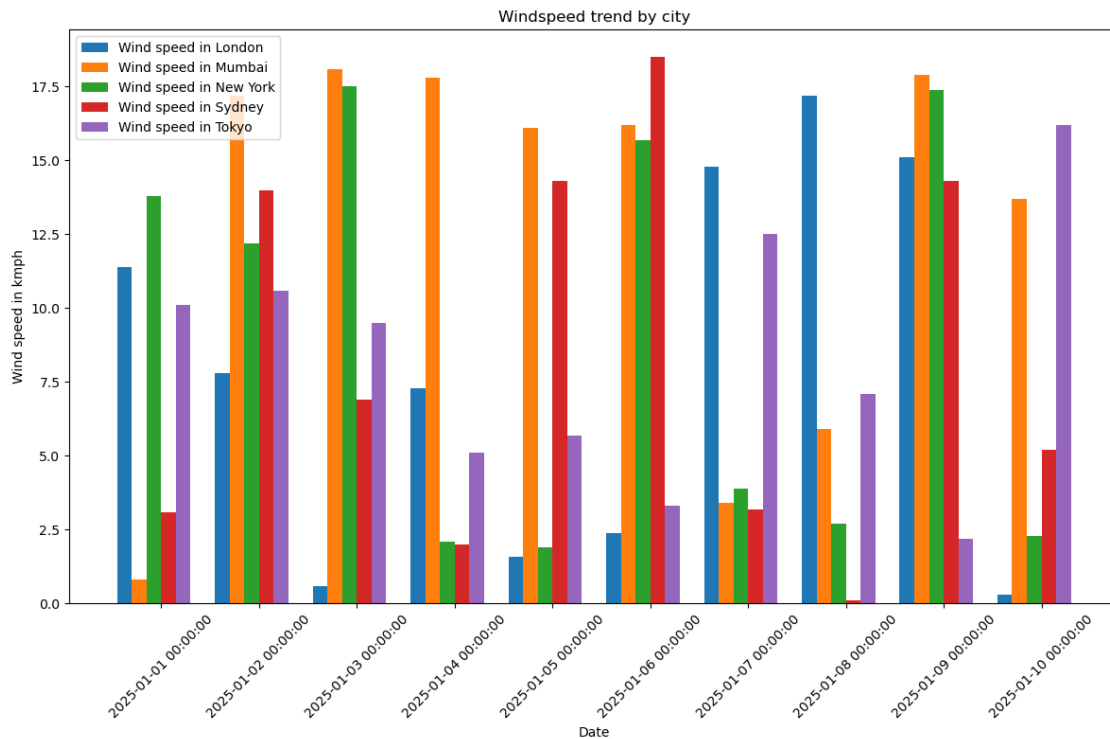
plt.figure(figsize = (12,8))
for i, (city,city_df) in enumerate(group_by_df2):
 plt.bar(
 x + i*width,
 city_df["WindSpeed_kmph"],
 width = width,
 label = f"Wind speed in {city}"
)

```

```

)
plt.title("Windspeed trend by city")
plt.xlabel("Date")
plt.ylabel("Wind speed in kmph")
plt.xticks(x+width*(len(group_by_df2)/2),dates,rotation=45)
plt.tight_layout()
plt.legend()
plt.show()

```



**Plot the Precipitation of all the cities using bar chart**

```

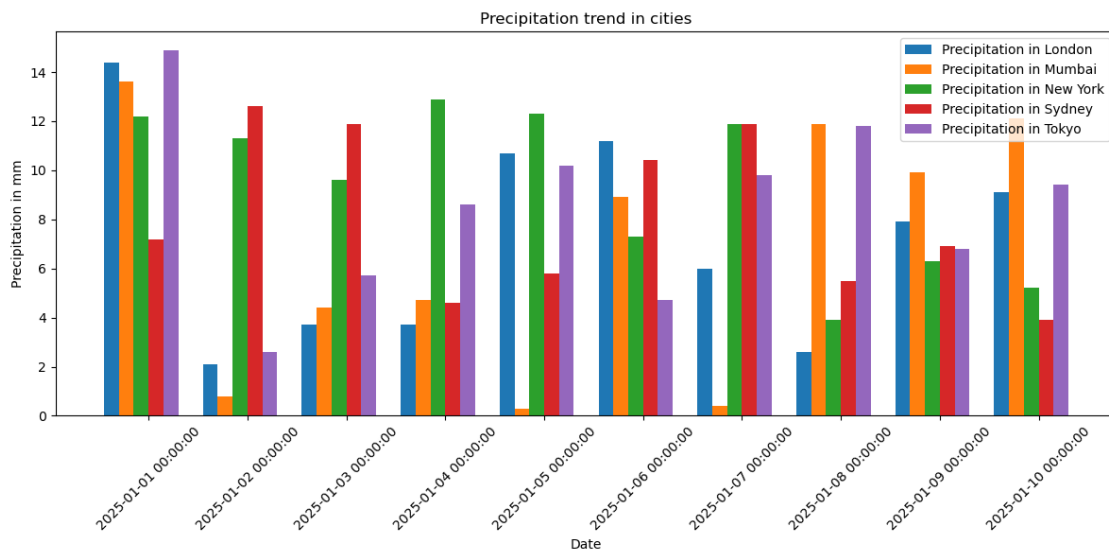
[65]: # get the unique dates and sort them chronologically
dates = sorted(df2["Date"].unique())
x = np.arange(len(dates))
width = 0.15

plt.figure(figsize = (12,6))
for i,(city,city_df) in enumerate(group_by_df2):
 plt.bar(
 x+i*width,
 city_df["Precipitation_mm"],
 width=width,
 label = f"Precipitation in {city}"
)

```



```
plt.title("Precipitation trend in cities")
plt.xlabel("Date")
plt.ylabel("Precipitation in mm")
plt.xticks(x+width*(len(group_by_df2)/2),dates,rotation=45)
plt.tight_layout()
plt.legend()
plt.show()
```



[ ]:

[ ]:

[ ]:

[ ]: